

ENTERPRISE REFERENCE GUIDE

AI, RAG & Agentic Systems Concept Reference

200 core concepts across 10 modules — foundations, mathematics, classical ML, deep learning, NLP, retrieval-augmented generation, agentic AI, LLMOps, infrastructure, and enterprise AI platforms — framed for both Data Scientists and Enterprise Architects.

Prepared for: Ram

Document date: August 2026

Scope: 10 Modules · 200 Concepts · Alphabetical Index

How to Use This Guide

This reference organizes 200 foundational-to-advanced AI concepts into ten modules, moving from theoretical bedrock (mathematics, classical machine learning) through modern deep learning and NLP, into the applied layers that matter most in production systems: retrieval-augmented generation, agentic AI, LLMOps, infrastructure, and enterprise AI platforms.

Each entry gives the single explanation needed to understand the concept: what it is, and the one or two things that matter most in practice — technically and at the platform/governance level — prioritized over exhaustive detail. Entries are kept short and dense on purpose, so the guide can be scanned quickly rather than read cover to cover.

Entries are grouped by module in the order they appear in the Table of Contents. Every term is also listed alphabetically in the Index at the back with a page reference. In the digital file, all Table of Contents and Index entries are clickable links that jump directly to the entry.

Element	Meaning
Module tag (e.g., RAG)	The module/domain this concept belongs to, matching the Table of Contents.
Explanation	One consolidated paragraph covering the definition and its most important practical implication.

Note on pagination: page numbers are estimates based on this document's layout and may shift slightly depending on your browser/printer. If you export to PDF via your browser's print function, enable "Headers and footers" for the browser's own running page numbers; internal links (Table of Contents and Index) will always jump to the correct entry regardless of the exact page count your PDF converter produces.

Table of Contents

10 modules · 20 concepts each · 200 total

How to Use This Guide	p. 2
Module 01 · AI Foundations	p. 4
Module 02 · Mathematics	p. 9
Module 03 · Classical Machine Learning	p. 14
Module 04 · Deep Learning	p. 19
Module 05 · Natural Language Processing	p. 24
Module 06 · Retrieval-Augmented Generation	p. 29
Module 07 · Agentic AI	p. 34
Module 08 · LLMOps	p. 39
Module 09 · Infrastructure	p. 44
Module 10 · Enterprise AI Platforms & Governance	p. 49
Appendix · Workflow, Tools & Keywords Matrix	p. 54
Alphabetical Index (A–Z)	p. 65

Page numbers reflect this document's fixed layout (5 pages per 20-concept module). See the pagination note on the previous page.

MODULE 01

AI Foundations

The conceptual bedrock of the field: what "intelligence" means for a machine, the core task types (classification, regression, clustering), and the reasoning/search/planning ideas that pre-date deep learning and still shape how agentic systems are designed today. Data scientists use this module to place a technique in the right category; enterprise architects use it to map business problems to the right AI capability tier and risk posture.

20 concepts

Artificial Intelligence FOUNDATIONS

The field of building systems that perform tasks — perception, reasoning, language, decision-making — that normally require human intelligence, spanning rule-based methods and modern statistical/neural approaches. Sets the top-level question of which AI capability tier (rules engine, ML model, LLM, or agent) fits a given process, and what risk regime applies.

Machine Learning FOUNDATIONS

A subset of AI where systems learn patterns from data rather than following hand-coded rules, improving as more examples are seen — the core toolkit for classification, regression, and clustering. Requires versioned data and models, retraining pipelines, and drift monitoring, a different operating model than traditional software.

Deep Learning FOUNDATIONS

A branch of ML using multi-layered neural networks to automatically learn hierarchical feature representations from raw data such as text, images, or audio, without manual feature engineering. Drives GPU/accelerator capacity planning, model-serving infrastructure, and build-vs-buy decisions more than classical ML.

Neural Network FOUNDATIONS

A computational model of interconnected layers of weighted nodes, trained via gradient-based optimization to approximate complex functions — the base architecture underlying nearly all modern deep learning. Model size and architecture choice directly determine inference cost, latency, and hardware requirements at scale.

Expert Systems FOUNDATIONS

Early rule-based AI programs that encode human expert knowledge as if-then rules with an inference engine, for reasoning in a narrow domain. Largely superseded by learned models for pattern-heavy tasks, but still useful where explainability and regulatory auditability outweigh statistical learning — e.g., eligibility rules.

Knowledge Representation FOUNDATIONS

Methods for encoding facts, relationships, and rules so a system can reason over them — ontologies, knowledge graphs, logic-based schemas — shaping how structured knowledge grounds an LLM's answers. Determines whether enterprise knowledge assets are reusable across AI applications, or re-built and siloed per project.

Search Algorithms FOUNDATIONS

Systematic methods — breadth-first, depth-first, A*, beam search — for exploring a space of possible states to find an optimal or acceptable solution, underpinning planning and decoding strategies like beam search. Still relevant to optimization and routing systems where search-based solvers outperform learned heuristics.

Reasoning FOUNDATIONS

The process of deriving conclusions, plans, or answers from available information using logical, probabilistic, or learned inference — a growing LLM focus via chain-of-thought and multi-step inference. Reasoning-capable models increase system autonomy, so multi-step decisions require stronger guardrails and audit trails.

Inference FOUNDATIONS

The stage where a trained model is applied to new input to produce an output, as opposed to training where parameters are learned — inference-time latency and cost per call are what production systems experience. Inference cost, not training cost, typically dominates the long-run total cost of ownership.

Optimization FOUNDATIONS

The mathematical process of finding parameter values that minimize or maximize an objective function, most commonly a loss function — the engine behind model training. Also applies at the system level: resource allocation, cost, and SLA trade-offs across a platform are optimization problems themselves.

Planning FOUNDATIONS

Generating a sequence of actions to reach a goal from a starting state — a classic AI problem now central to agentic LLM systems, decomposing a task into ordered, verifiable steps. Planning-capable agents introduce new failure modes (wrong plans, runaway loops) requiring explicit approval gates and cost ceilings.

Learning FOUNDATIONS

The general capacity of a system to improve behavior from experience or data — supervised, unsupervised, reinforcement, and self-supervised paradigms, chosen based on what labels or feedback signals are available. Different paradigms imply different data pipelines and compliance considerations, especially with live user feedback.

Prediction FOUNDATIONS

The output a trained model produces for a given input — an estimated label, value, or probability distribution — the core deliverable of most ML systems. Prediction confidence must be surfaced to downstream systems and users so decisions aren't made on unchecked output.

Classification FOUNDATIONS

A supervised learning task that assigns an input to one of a fixed set of discrete categories, such as spam vs. not-spam, evaluated with accuracy, precision, recall, and F1. In regulated domains (credit, hiring, healthcare) it often requires explainability and fairness audits before production use.

Regression FOUNDATIONS

A supervised learning task that predicts a continuous numeric value, such as price or demand, rather than a discrete category, evaluated with error-based metrics like RMSE or MAE. Regression outputs commonly feed downstream business logic, so error bounds matter as much as the point estimate.

Clustering FOUNDATIONS

An unsupervised learning technique that groups similar data points together without predefined labels, used for segmentation and exploratory analysis when labeled data doesn't exist. Powers customer segmentation use cases, but cluster definitions drift over time and need periodic revalidation.

Anomaly Detection FOUNDATIONS

Techniques for identifying data points or events that deviate significantly from expected patterns — used in fraud, security, and quality monitoring via statistical thresholds and unsupervised models. A core building block for AI-driven security operations, where false positive/negative trade-offs carry real operational cost.

Recommendation FOUNDATIONS

Systems that predict which items — products, content, actions — a user is likely to prefer, using collaborative, content-based, or hybrid methods. Requires real-time feature pipelines and A/B testing platforms, since quality is judged by engagement and business metrics, not accuracy alone.

Decision Support FOUNDATIONS

AI systems designed to augment human decision-making by surfacing relevant information, predictions, or options rather than acting autonomously, presenting output with context and uncertainty. Often the safer default for high-stakes processes — keeping a human in the loop while capturing AI's analytical leverage.

Automation FOUNDATIONS

The use of AI and rules-based systems to execute tasks or processes with minimal human intervention, with candidates identified by task repetitiveness, data availability, and error tolerance. Automation scope maps directly to risk appetite — full automation, human-in-the-loop, or human-on-the-loop are architectural decisions.

MODULE 02

Mathematics

The quantitative language underneath every model: linear algebra and calculus for how models represent and learn, probability and statistics for how they handle uncertainty, and graph/set/combinatorial tools that show up in everything from knowledge graphs to retrieval scoring. Data scientists use this module day to day when debugging models; enterprise architects need enough of it to sanity-check vendor claims and understand where numerical instability or bias can enter a system.

20 concepts

Linear Algebra MATHEMATICS

The branch of mathematics dealing with vectors, matrices, and linear transformations — the structural language of neural networks and embeddings, since every layer is a matrix multiplication followed by a nonlinearity. Matrix and tensor throughput is the reason AI workloads need GPUs rather than general-purpose CPUs.

Calculus MATHEMATICS

The mathematics of continuous change, used in ML primarily for the derivatives and gradients that drive optimization — backpropagation is repeated application of the chain rule to compute how loss changes with each weight. Understanding gradients helps explain why training can be unstable or slow.

Probability MATHEMATICS

The mathematical framework for quantifying uncertainty and the likelihood of events, foundational to nearly all ML outputs — model outputs are usually probability distributions rather than hard answers. Confidence scores and thresholds, not just predictions, should drive downstream automation decisions and risk tiers.

Statistics MATHEMATICS

Methods for collecting, analyzing, and drawing conclusions from data, including hypothesis testing, sampling, and estimation, underpinning experiment design and A/B testing. Statistical rigor in evaluation prevents shipping AI changes based on spurious or non-significant results.

Vectors MATHEMATICS

Ordered lists of numbers representing a point or direction in space — text, images, and users are all converted into vectors so similarity and distance can be computed mathematically. Vector representations are the common currency across search, recommendation, and RAG systems.

Matrices MATHEMATICS

Two-dimensional arrays of numbers used to represent linear transformations, datasets, and neural network weights — matrix multiplication is the dominant computation in both training and inference. Matrix size (parameter count) is a direct proxy for a model's memory footprint and hardware cost.

Eigenvalues MATHEMATICS

Special scalars associated with a matrix that describe how it stretches or compresses space along particular directions, central to PCA and other dimensionality-reduction techniques. Eigen-decomposition-driven reduction can cut storage and compute costs for large embedding systems.

Gradients MATHEMATICS

Vectors of partial derivatives indicating the direction of steepest increase of a function — the signal gradient descent uses to iteratively reduce a model's loss. Gradient instability (vanishing/exploding) is a common root cause of failed or stalled large-model training runs.

Bayes Theorem MATHEMATICS

A formula for updating the probability of a hypothesis as new evidence becomes available, relating prior, likelihood, and posterior probabilities — the basis of Naive Bayes and Bayesian inference. The conceptual foundation for how systems should revise confidence as new data or user feedback arrives.

Entropy MATHEMATICS

A measure of uncertainty or randomness in a probability distribution, used extensively in loss functions — cross-entropy loss is the standard objective for training classifiers and language models. High output entropy in a generative model is a useful signal for flagging low-confidence, unreliable responses.

Optimization MATHEMATICS

The formal study of finding the best solution from a set of feasible options according to an objective function and constraints, framing both model training and resource-allocation problems. Provides the formal language for balancing cost, latency, and accuracy trade-offs across an AI platform.

Convexity MATHEMATICS

A property of functions where any line segment between two points on the curve lies above it, guaranteeing a single global minimum — convex losses (linear/logistic regression) are easier to optimize reliably than non-convex deep-learning losses. Non-convexity explains why training outcomes vary run to run.

Markov Chains MATHEMATICS

A stochastic model describing a sequence of states where the next state depends only on the current one, not the full history, used in sequence modeling and reinforcement learning. A conceptual model for state-machine-like workflow systems, including simple agent transition logic.

Graph Theory MATHEMATICS

The mathematical study of networks of nodes and edges, used to model relationships between entities — underpinning knowledge graphs, graph neural networks, and multi-agent communication topologies. How enterprises model relationships between customers, products, and documents for graph-based RAG.

Set Theory MATHEMATICS

The mathematical study of collections of distinct objects and the relationships (union, intersection, membership) between them, providing the logical basis for data filtering and metadata-based retrieval filters. Access-control and data-partitioning logic is fundamentally set-based reasoning.

Combinatorics MATHEMATICS

The mathematics of counting, arrangement, and combination of discrete objects, relevant to hyperparameter search spaces and beam search branching in decoding. Explains why exhaustive search over configuration or agent-action spaces quickly becomes computationally infeasible at scale.

Differential Equations MATHEMATICS

Equations relating a function to its derivatives, used to describe systems that evolve continuously over time — the theoretical basis of diffusion models, which reverse a differential noise process. Relevant mainly when evaluating generative image/video vendors whose models are diffusion-based.

Numerical Methods MATHEMATICS

Algorithms for approximating solutions to mathematical problems that cannot be solved exactly, including the stability techniques (gradient clipping, mixed precision) practical training relies on. Numerical precision choices (FP32, FP16, INT8) directly trade off model accuracy against inference cost and speed.

Tensor Algebra MATHEMATICS

The generalization of vectors and matrices to higher-dimensional arrays (tensors), the native data structure of deep learning frameworks — all data is represented and manipulated as tensors during training and inference. Tensor shapes and batch sizes are key levers for tuning GPU utilization.

Random Variables MATHEMATICS

Variables whose possible values are outcomes of a random phenomenon, described by a probability distribution — the formal object behind model outputs and sampling temperature in generation. Explains why identical prompts to a generative model can yield different outputs, a factor in testing and QA strategy.

MODULE 03

Classical Machine Learning

The pre-deep-learning-era toolkit that still runs a large share of production systems: tree ensembles for tabular data, distance-based and probabilistic classifiers, unsupervised clustering, and the model-development discipline (feature engineering, validation, tuning, bias/variance) that applies to every model regardless of architecture. For data scientists this is the everyday toolkit; for enterprise architects it's often the cheapest, most explainable, and most maintainable option.

20 concepts

Decision Tree ML

A model that splits data recursively on feature thresholds to form a tree of decisions leading to a prediction — simple and interpretable, though prone to overfitting if grown without constraints. Its interpretability makes it attractive for regulated decisions requiring a clear audit trail.

Random Forest ML

An ensemble of many decision trees trained on random data/feature subsets, with predictions averaged or voted across trees — a strong, low-effort baseline that resists overfitting better than a single tree. A dependable workhorse for internal analytics tools where model risk needs to stay low.

XGBoost ML

A highly optimized gradient-boosted tree library that builds trees sequentially, each correcting the errors of the previous ones — often the top performer on structured/tabular data. A mature, widely supported library with strong tooling for deployment, monitoring, and explainability (e.g., SHAP).

LightGBM ML

A gradient boosting framework optimized for speed and memory efficiency on large tabular datasets using histogram-based splitting, preferred over XGBoost when training speed and dataset size become bottlenecks. Lower training compute cost makes it attractive for frequent retraining pipelines.

CatBoost ML

A gradient boosting library with native, efficient handling of categorical features without extensive preprocessing, reducing feature-engineering overhead on datasets with many categorical columns. Cuts pipeline complexity for teams working directly with raw categorical business data like CRM/ERP.

SVM ML

Support Vector Machine — a classifier that finds the hyperplane maximizing the margin between classes, optionally using kernels for non-linear boundaries. Effective on smaller, high-dimensional datasets, though largely overtaken by tree ensembles and deep learning at scale; still useful for lightweight, resource-constrained tasks.

KNN ML

K-Nearest Neighbors — a non-parametric method that classifies a point based on the majority label of its closest neighbors in feature space, simple but slow at inference on large datasets without indexing. The conceptual ancestor of the vector-search retrieval used in modern RAG systems.

Naive Bayes ML

A probabilistic classifier applying Bayes theorem with a "naive" assumption of feature independence — fast, simple, and surprisingly effective for text classification and spam filtering. A lightweight option when compute budgets are tight and approximate accuracy is acceptable.

Logistic Regression ML

A linear model that estimates the probability of a binary outcome using a sigmoid function over a weighted sum of features — the standard interpretable baseline for binary classification. Frequently the model of choice in regulated industries because its coefficients are straightforward to explain to auditors.

Linear Regression ML

A model that predicts a continuous outcome as a weighted linear combination of input features — the foundational regression baseline, valued for interpretability and low cost. Often the first model deployed for forecasting use cases before investing in more complex architectures.

PCA ML

Principal Component Analysis — a dimensionality-reduction technique that projects data onto directions of maximum variance, used to compress features, visualize high-dimensional data, and reduce noise. Reduces storage and compute footprint for large feature sets or embeddings while preserving most of the signal.

KMeans ML

An unsupervised clustering algorithm that partitions data into K groups by minimizing distance to cluster centroids — a fast, simple starting point for segmentation, sensitive to the chosen K. Widely used for customer and usage segmentation where speed outweighs the need for perfect clusters.

DBSCAN ML

A density-based clustering algorithm that groups closely packed points and marks sparse points as outliers, without needing a predefined cluster count — useful when cluster shapes are irregular. A natural fit for anomaly and fraud detection pipelines, since outliers are a built-in output.

Hierarchical Clustering ML

A clustering method that builds a tree (dendrogram) of nested clusters by merging or splitting groups progressively, useful when relationships between clusters at different granularities matter. Supports building taxonomy-like structures for organizing content or product catalogs.

Feature Engineering ML

Transforming raw data into input variables (features) that better represent the underlying problem to a model — often the single highest-leverage activity in classical ML. Feature pipelines need the same governance as data pipelines: lineage, versioning, and training/serving consistency.

Feature Selection ML

Choosing the most relevant subset of features to use, removing redundant or noisy ones, improving generalization, training speed, and interpretability. Fewer features also simplify data governance and reduce the surface area for sensitive-data exposure.

Cross Validation ML

A model evaluation technique that splits data into multiple train/test folds to get a more reliable estimate of generalization performance — the standard defense against overly optimistic single-split estimates. Reported model metrics should always be cross-validated before justifying a production go-live decision.

Hyperparameter Tuning ML

Searching for the best configuration values (learning rate, tree depth, etc.) set before training rather than learned, via grid, random, or Bayesian search. Tuning budgets should be capped and automated — uncontrolled search can silently consume large compute budgets.

Bias ML

In the statistical sense, the systematic error a model makes from overly simplistic assumptions (underfitting); in the fairness sense, unequal treatment across groups. Fairness bias carries legal and reputational risk and typically requires a formal review process before deployment.

Variance ML

The sensitivity of a model's predictions to small fluctuations in the training data — high variance signals overfitting, managed through regularization, more data, or simpler models. High-variance models behave unpredictably in production when real-world data drifts from the training distribution.

MODULE 04

Deep Learning

The architectures and training mechanics behind modern AI: convolutional and recurrent networks, the transformer and attention mechanism that power today's language models, and the training tricks (normalization, dropout, residual connections) that make deep networks trainable at all. Data scientists need this to choose and debug architectures; enterprise architects need it to understand why certain models need specific hardware, memory, and training infrastructure.

20 concepts

CNN DEEP LEARNING

Convolutional Neural Network — applies sliding filters over input data to detect local patterns, historically dominant in computer vision due to weight sharing and translation invariance. Still the backbone of many production vision systems (quality inspection, document scanning) despite transformer-based alternatives.

RNN DEEP LEARNING

Recurrent Neural Network — processes sequences step by step, maintaining a hidden state that carries information forward. Largely superseded by transformers for language due to difficulty training on long sequences; legacy RNN-based systems are common migration targets today.

LSTM DEEP LEARNING

Long Short-Term Memory — an RNN variant with gating mechanisms that let it retain or forget information over longer sequences, solving much of the vanishing-gradient problem. Still cost-effective for smaller-scale time-series and sequence tasks where a full transformer is unnecessary overhead.

GRU DEEP LEARNING

Gated Recurrent Unit — a simplified alternative to LSTM with fewer gates, offering similar benefits at lower computational cost. A pragmatic choice for resource-constrained, on-device, or edge sequence-modeling applications.

Transformer DEEP LEARNING

The neural architecture built entirely on attention mechanisms (no recurrence), enabling parallel processing of sequences and underlying nearly all modern LLMs. Transformer scale — parameters and context length — is the primary driver of GPU memory and cost requirements across the AI stack.

Attention DEEP LEARNING

A mechanism that lets a model weigh the relevance of different parts of the input when producing each part of the output, letting transformers relate distant tokens directly. Its compute cost scales with sequence length squared, directly shaping context-window pricing and latency at longer inputs.

Self Attention DEEP LEARNING

A specific form of attention where a sequence attends to itself, letting each token incorporate context from every other token — the mechanism giving transformers their contextual understanding. Explains why longer documents cost more to process, since cost grows quadratically with input length.

Positional Encoding DEEP LEARNING

A technique for injecting information about token order into transformer inputs, since attention itself has no inherent sense of sequence position. The choice of scheme (learned, sinusoidal, rotary) underpins claims about a model's usable context window — a key spec for long-document use cases.

Batch Normalization DEEP LEARNING

A technique that normalizes layer inputs across a mini-batch during training, stabilizing and speeding up convergence by reducing sensitivity to initialization and learning rate. A training-stability detail relevant when a custom-trained model underperforms a vendor's tuned baseline.

Dropout DEEP LEARNING

A regularization technique that randomly disables a fraction of neurons during training to prevent overfitting, especially useful on smaller or noisier datasets. Part of the standard toolkit that should be present in any custom model-training pipeline.

Residual Network DEEP LEARNING

An architecture pattern (ResNet) that adds shortcut "skip connections" around layers, letting gradients flow more easily through very deep networks and enabling much deeper networks than previously possible. Now a default building block in transformer architectures too.

Autoencoder DEEP LEARNING

A neural network trained to reconstruct its input after compressing it through a lower-dimensional bottleneck layer, used for dimensionality reduction, denoising, and anomaly detection via reconstruction error. A cost-effective anomaly-detection pattern for fraud or equipment-failure monitoring without labeled data.

GAN DEEP LEARNING

Generative Adversarial Network — two networks (generator and discriminator) trained in competition, where the generator learns to produce increasingly realistic synthetic data. Historically the leading approach for image synthesis, and still useful for augmenting scarce training data while reducing exposure of sensitive real records.

Diffusion DEEP LEARNING

A generative modeling approach that learns to reverse a gradual noising process, producing data (typically images) from random noise step by step. Currently the state of the art for image and video generation, though compute-intensive relative to a single LLM call.

Vision Transformer DEEP LEARNING

An adaptation of the transformer architecture to images, treating patches of an image as a sequence of tokens — increasingly competitive with or superior to CNNs given enough training data. Enables a unified transformer-based platform strategy across text, vision, and multimodal workloads.

Activation Function DEEP LEARNING

A nonlinear function (ReLU, GELU, sigmoid) applied after each layer's linear operation, giving neural networks the ability to model complex, non-linear relationships. A low-level detail generally abstracted away by frameworks and pretrained model choices.

Backpropagation DEEP LEARNING

The algorithm that computes gradients of the loss with respect to every weight in a network by applying the chain rule backward through the layers. The core training mechanism for essentially all neural networks, and the reason training requires substantial, often specialized, GPU infrastructure.

Gradient Descent DEEP LEARNING

An iterative optimization algorithm that updates model parameters in the direction that reduces the loss function, using the gradient from backpropagation. Training time and cost scale directly with the number of steps needed to reach acceptable performance.

Adam Optimizer DEEP LEARNING

A widely used gradient descent variant that adapts the learning rate per parameter using estimates of past gradients, delivering fast, robust convergence with minimal tuning. The default, safe optimizer choice that reduces the risk of failed training runs from poorly chosen learning rates.

Loss Function DEEP LEARNING

A function that measures the difference between a model's predictions and the true target, providing the signal that training tries to minimize. Its choice directly encodes what "good performance" means; a mismatched loss function is a common, easy-to-miss reason an "accurate" model fails to deliver business value.

MODULE 05

Natural Language Processing

The path from raw text to modern large language models: classical text-processing steps (tokenization, NER, parsing), the embedding models that turned words into vectors, the landmark pretrained architectures (BERT, GPT, T5), and the adaptation techniques — prompting, instruction tuning, RLHF, LoRA — that turn a general-purpose LLM into a task-specific enterprise tool.

20 concepts

Tokenization NLP

The process of splitting text into smaller units (tokens) — words, subwords, or characters — that a model can process numerically. Tokenizer choice affects vocabulary size and rare-word handling, and since LLM pricing is typically per token, tokenization efficiency directly affects the cost of every API call.

Stemming NLP

A crude, rule-based technique for reducing words to a root form by stripping suffixes (e.g., "running" to "run"). A lightweight legacy preprocessing step, largely unnecessary with modern embedding models and worth retiring when modernizing older enterprise search systems.

Lemmatization NLP

A linguistically informed technique for reducing words to their dictionary base form, more accurate than stemming. Still useful in compliance and legal text-processing pipelines that need precise, rule-based text normalization.

NER NLP

Named Entity Recognition — identifying and classifying entities in text such as people, organizations, dates, or locations, now often handled by fine-tuned or prompted LLMs. Powers automated PII redaction and structured data extraction from unstructured documents, a common compliance use case.

POS Tagging NLP

Part-of-Speech Tagging — labeling each word in a sentence with its grammatical role, a building block for classical parsing now mostly implicit inside modern language models. Rarely a standalone concern today, but still underlies some legacy grammar and compliance-checking tools in production.

Dependency Parsing NLP

Analyzing the grammatical structure of a sentence to identify relationships between words, such as subject-verb-object links. Useful for precise information extraction where relationships between entities matter, still valuable in contract-analysis and legal-tech pipelines.

Word2Vec NLP

An early neural technique for learning dense word embeddings by predicting a word from its context, introducing the idea that word meaning could be captured geometrically. Largely superseded by contextual transformer embeddings, but historically important for understanding the embedding paradigm.

FastText NLP

An extension of Word2Vec that represents words as bags of character n-grams, enabling better handling of rare and misspelled words. A low-cost option for lightweight, fast text classification where full LLM inference would be overkill.

BERT NLP

Bidirectional Encoder Representations from Transformers — a pretrained model reading text in both directions to build rich contextual representations, excellent for understanding tasks (classification, NER, embeddings) but not open-ended generation. Often the right, lower-cost choice when a full generative LLM isn't needed.

RoBERTa NLP

A robustly optimized version of BERT trained with more data, longer training, and refined hyperparameters for improved performance. A common drop-in upgrade from BERT, and a good illustration that training-recipe improvements alone can meaningfully boost model quality.

T5 NLP

Text-to-Text Transfer Transformer — frames every NLP task (translation, summarization, classification) as converting input text to output text, simplifying multi-task model design with one consistent format. Demonstrates the shift toward general-purpose text-in/text-out architectures that later scaled into today's LLMs.

GPT NLP

Generative Pre-trained Transformer — a decoder-only transformer trained to predict the next token, forming the basis of modern conversational and generative LLMs. Which GPT-class model family, hosted where, under what data and pricing terms, is the central platform decision anchoring most LLM strategy.

Sentence Embeddings NLP

Dense vector representations of entire sentences or passages capturing semantic meaning for comparison and search — the core representation behind semantic search, clustering, and RAG. Embedding model choice and consistency between indexing and querying is a critical, easy-to-get-wrong design decision.

Prompt Engineering NLP

Designing inputs to an LLM — instructions, examples, formatting — to reliably elicit the desired output without changing model weights. Often the fastest, cheapest lever for improving output quality before fine-tuning, and prompts are production artifacts needing version control like any other application logic.

Instruction Tuning NLP

Fine-tuning a base language model on examples of instructions paired with desired responses, turning a raw next-token predictor into a usable assistant. Explains the difference between "base" and "instruct/chat" model variants — the instruct version is almost always the right default.

RLHF NLP

Reinforcement Learning from Human Feedback — uses human preference judgments and a learned reward model to further align outputs with desired behavior beyond instruction tuning. A key reason different vendor models "feel" different in tone and safety behavior despite similar architectures.

LoRA NLP

Low-Rank Adaptation — a parameter-efficient fine-tuning method that trains small low-rank matrices instead of updating all model weights, making custom fine-tuning practical on modest hardware. Enables cost-effective, per-customer or per-domain model customization without maintaining full separate model copies.

QLoRA NLP

A variant of LoRA that fine-tunes a quantized (compressed) base model, further reducing memory requirements — allowing large models to be fine-tuned on a single GPU instead of a multi-GPU cluster. Lowers the infrastructure bar for in-house fine-tuning capability.

PEFT NLP

Parameter-Efficient Fine-Tuning — the umbrella term for techniques like LoRA and adapters that tune a small subset of parameters rather than the full model. The default approach to customization now, reducing compute cost and the risk of catastrophic forgetting versus full fine-tuning.

Fine Tuning NLP

Continuing to train a pretrained model on a smaller, task- or domain-specific dataset to specialize its behavior, chosen when prompting alone can't reliably achieve consistent, deeply specialized results. A build-vs-configure decision that adds real MLOps overhead — versioning, retraining, evaluation — prompting avoids.

MODULE 06

Retrieval-Augmented Generation

The architecture pattern that grounds LLM outputs in external, up-to-date, enterprise-controlled data instead of relying solely on parametric memory: retrieval methods (sparse, dense, hybrid), query and result refinement techniques, and the emerging family of self-correcting and agentic RAG variants. This module is the technical core of most enterprise LLM deployments today.

20 concepts

Semantic Search RAG

Search that matches queries and documents by meaning, using vector embeddings and similarity metrics, rather than exact keyword overlap. The foundational capability that separates a modern knowledge platform from a legacy keyword-search intranet tool, evaluated with recall/precision-at-k.

BM25 RAG

A classic sparse, keyword-based ranking function that scores documents by term frequency and rarity, still a strong retrieval baseline. Excels at exact-match terms (codes, IDs, names) embeddings can miss, and is a cheap, well-understood safety net rather than a sole retrieval method.

Hybrid Search RAG

A retrieval strategy that combines sparse (keyword) and dense (semantic) search results, typically merged with a weighted or learned scoring function. Consistently outperforms either method alone across most enterprise document sets — the recommended default retrieval architecture.

Dense Retrieval RAG

Retrieval that represents queries and documents as dense embedding vectors and ranks results by vector similarity (e.g., cosine distance), capturing semantic and paraphrase matches keyword search misses. Requires vector database infrastructure and re-embedding whenever the underlying embedding model changes.

Sparse Retrieval RAG

Retrieval based on high-dimensional, mostly-zero vectors representing exact term matches, as in traditional inverted-index search engines — more interpretable and precise for exact-match queries, but lacking semantic generalization. Can often reuse existing enterprise search infrastructure rather than requiring new vector infrastructure.

ColBERT RAG

A retrieval model that computes fine-grained, token-level embeddings and matches queries to documents via "late interaction" scoring, delivering higher precision than single-vector dense retrieval. Comes with higher storage and compute cost — a precision-vs-cost trade-off to evaluate per use case.

Parent Child Retrieval RAG

A chunking strategy that retrieves small, precise child chunks for matching but returns their larger parent chunk or document for full context, balancing retrieval precision with generation quality. A standard pattern to adopt when RAG answers are technically relevant but feel context-poor to users.

Multi Query Retrieval RAG

A technique where an LLM generates several reformulated versions of a user's query, retrieves results for each, and merges them — improving recall for ambiguous or poorly phrased queries. Increases quality at the cost of extra LLM calls and latency, a trade-off to tune per use case.

Multi Vector Retrieval RAG

Representing a single document with multiple embeddings — per chunk, summary, or hypothetical question — to improve the chance a relevant document is retrieved when a query matches a specific detail. Increases index storage proportionally to vectors per document, a capacity-planning factor.

Context Compression RAG

Techniques for filtering or summarizing retrieved content before passing it to the LLM, keeping only what's relevant to reduce noise that can degrade answer quality. Directly reduces token cost per query, which matters at scale with large context windows and document sets.

HyDE RAG

Hypothetical Document Embeddings — an LLM first generates a hypothetical answer, which is then embedded and used to retrieve similar real documents. Can improve retrieval when a question's wording differs significantly from source phrasing, at the cost of an extra LLM call in the retrieval path.

Query Expansion RAG

Automatically adding related terms, synonyms, or context to a user's query before retrieval to broaden the match surface. A lightweight, low-cost recall-improvement lever worth trying before investing in more expensive retrieval architecture changes.

Metadata Filtering RAG

Narrowing retrieval to documents matching structured attributes (date, department, access level) before or alongside semantic ranking. The primary mechanism for enforcing data access control and permissions within a RAG system — a non-negotiable for enterprise deployments.

Recursive Retrieval RAG

A retrieval process that iteratively refines its search — retrieving, evaluating results, and retrieving again — useful for complex, multi-hop questions needing information from several sources. Increases latency and cost proportionally to retrieval rounds, so needs explicit iteration limits.

Re Ranking RAG

A second-stage scoring step that reorders an initial set of retrieved candidates using a more accurate (often cross-encoder) model, correcting the imprecision of fast first-stage retrieval. A high-ROI addition to most RAG pipelines for modest added latency and cost.

Knowledge Graph RAG RAG

A RAG variant that retrieves structured facts and relationships from a knowledge graph, alongside or instead of unstructured text chunks, improving accuracy on relationship and multi-hop questions. Leverages existing enterprise data models (master data, ontologies) as a first-class AI input, not just documents.

Self RAG RAG

A RAG approach where the model critiques its own retrieved evidence and generated answer, deciding whether to retrieve more or revise its response. Reduces hallucination via explicit self-verification, trading added latency and compute for measurably higher reliability.

CRAG RAG

Corrective RAG — evaluates the quality of retrieved documents and triggers fallback actions (web search, query rewrite) when they're insufficient. A resilience pattern worth adopting when the internal knowledge base has known coverage gaps.

Adaptive RAG RAG

A RAG strategy that dynamically chooses how much retrieval — or none at all — a query needs based on its complexity, avoiding unnecessary retrieval for simple queries. Reduces average cost per query across a mixed workload compared to always running the full pipeline.

Agentic RAG RAG

A RAG architecture where an agent plans and orchestrates multiple retrieval and reasoning steps, choosing tools and sources dynamically instead of following a fixed pipeline. The most powerful but least predictable RAG pattern — needs strong observability, cost controls, and guardrails before production use.

MODULE 07

Agentic AI

The design patterns for systems that don't just answer a single question but plan, act, use tools, and coordinate across multiple steps or multiple specialized agents. This module covers the functional roles (planner, executor, critic), coordination patterns (multi-agent, swarm, router), and application archetypes (browser, code, research agents) that make up modern agentic architectures — along with the human-oversight patterns needed to keep them safe.

20 concepts

Planner AGENTIC AI

The component of an agentic system responsible for decomposing a high-level goal into an ordered sequence of executable steps, typically an LLM prompted to reason step by step before any action is taken. Plan quality directly determines system reliability — plans should be inspectable and, for risky actions, approvable before execution.

Executor AGENTIC AI

The component that carries out individual steps of a plan, typically by invoking tools, APIs, or sub-agents. This is where real-world side effects happen, so it needs the tightest permissioning and audit logging in the system, and benefits from being tested and constrained independently of planning.

Supervisor AGENTIC AI

An orchestrating agent that oversees and directs one or more worker agents, deciding what happens next based on their outputs. Centralizes control and observability in multi-agent systems, making them easier to monitor than fully decentralized designs.

Reflection AGENTIC AI

A technique where an agent reviews and critiques its own prior output or actions before finalizing a response or proceeding further. Adds latency and cost per task but measurably reduces error rates — worth it for higher-stakes agent tasks.

Memory AGENTIC AI

Mechanisms for an agent to retain and recall information across steps or sessions, from short-term context to long-term persistent stores, distinguishing stateless systems from ones that build on prior interactions. Persistent agent memory raises real data-retention and privacy questions that need explicit policy before deployment.

Tool Calling AGENTIC AI

The capability of an LLM to invoke external functions, APIs, or systems in a structured way as part of generating its response — the mechanism that turns a text generator into an agent capable of taking real actions. Every tool exposed to a model is a new attack surface and permission boundary that must be explicitly scoped and secured.

Multi Agent AGENTIC AI

An architecture where multiple specialized agents collaborate, each handling a distinct sub-task or role, rather than one agent doing everything. Specialization often improves reliability over a single generalist agent, but increases system complexity and inter-agent failure modes — needs strong tracing to debug.

Coordinator AGENTIC AI

A role or component responsible for sequencing and synchronizing the work of multiple agents toward a shared goal — managing hand-offs, shared state, and conflict resolution. Analogous to an orchestration layer in distributed systems, and the natural place to enforce cross-agent policy.

Router AGENTIC AI

A component that directs an incoming request to the appropriate agent, model, or tool based on its content or intent, usually a lightweight, fast classification step. A key cost-control lever — routing simple queries to cheaper models and reserving expensive agents for complex cases.

Critic AGENTIC AI

An agent or model role that evaluates the output or actions of another agent against quality, correctness, or safety criteria, used in reflection and multi-agent loops to catch errors before they propagate. Provides a checkpoint for automated quality control, reducing reliance on manual human review of every output.

Judge AGENTIC AI

An LLM used specifically to score or compare outputs against a rubric — "LLM-as-judge" — rather than to execute tasks, widely used to scale qualitative evaluation where human review doesn't scale. Judge-based evaluation itself needs calibration and periodic human spot-checks; it's a useful proxy, not a perfect one.

Swarm AGENTIC AI

A multi-agent pattern with many lightweight, loosely coordinated agents working in parallel, often without a central controller — useful for parallelizable exploration like searching many sources. Harder to govern than supervisor-based architectures, since cost and behavior can be less predictable at scale.

Workflow AGENTIC AI

A defined, often deterministic sequence of steps — which may include agent or LLM calls — that a system follows to complete a task. A more constrained, predictable alternative to fully autonomous agents, and generally the safer starting point for production automation.

Human in the Loop AGENTIC AI

A design pattern where a human reviews, approves, or corrects an agent's output or actions at defined checkpoints before they take effect. The standard risk-mitigation pattern for deploying agentic AI into high-stakes or irreversible-action processes, while also generating valuable feedback data.

Autonomous Agent AGENTIC AI

An agent capable of planning and executing multi-step tasks toward a goal with minimal or no human intervention during execution — the highest-autonomy point on the spectrum. Carries the highest operational risk of any pattern in this module, requiring cost ceilings, timeouts, and kill switches by design.

Reasoning Agent AGENTIC AI

An agent architecture that explicitly interleaves reasoning steps (thinking) with actions (tool calls), often following a "reason-then-act" loop. Making intermediate reasoning explicit and inspectable improves task success on complex problems, and the exposed traces are valuable for debugging and compliance review.

Planning Agent AGENTIC AI

An agent specialized in producing and refining multi-step plans, often as a distinct role from the agents that execute those plans. Separating this role allows the plan to be reviewed or validated independently — a natural point to insert human approval before any action-taking agent is triggered.

Browser Agent AGENTIC AI

An agent that perceives and interacts with web pages — reading content, clicking, filling forms — to complete tasks requiring the live web, combining DOM/vision understanding with action-taking. Carries elevated risk (unverified links, credential use, unintended submissions) requiring strict domain allow-lists and human review of sensitive actions.

Code Agent AGENTIC AI

An agent that writes, runs, and debugs code autonomously to accomplish a task, typically operating in a sandboxed execution environment. A high-value application for engineering productivity, but requires strict sandboxing, dependency controls, and code review gates before generated code reaches production systems.

Research Agent AGENTIC AI

An agent that autonomously searches, reads, and synthesizes information from multiple sources to answer a complex, open-ended question, iterating until it has gathered sufficient evidence. Source citation and traceability are essential so research outputs can be verified before being used for decisions.

MODULE 08

LLMOps

The operational discipline for running LLM and agentic systems reliably in production: versioning and evaluating prompts and models, tracing and observability for debugging non-deterministic systems, safety controls (guardrails, hallucination detection, PII masking), and the deployment practices (canary, rollback, rate limiting) adapted from traditional DevOps to the specific failure modes of generative AI. This is the module enterprise architects lean on most when defining platform standards.

20 concepts

Prompt Versioning LLMOPS

Tracking changes to prompts over time with version history, similar to source code, so behavior changes can be traced and rolled back. Prompts are production configuration, not throwaway text — they need the same change-management rigor as code, including comparing versions against consistent evaluation sets before promoting a change.

Model Registry LLMOPS

A centralized system for storing, versioning, and tracking metadata about trained models across their lifecycle. It provides reproducibility — knowing exactly which model version, training data, and parameters produced a given result — and is a prerequisite for audit and compliance.

Tracing LLMOPS

Recording the detailed step-by-step execution path of a request through an LLM or agent system — prompts, tool calls, retrievals, and outputs. The primary debugging tool for understanding why an agent produced an unexpected result; without it, agentic system failures are effectively unauditable.

Observability LLMOPS

The broader practice of instrumenting a system with logs, metrics, and traces so its internal behavior can be understood from the outside — quality, cost, latency, and drift signals over time. A non-negotiable platform capability before scaling any LLM system beyond a pilot.

Evaluation LLMOPS

The systematic measurement of model or system quality against defined metrics, test sets, or rubrics, combining automated metrics, LLM-as-judge scoring, and human review. Should be a continuous production process, not a one-time pre-launch check, since models and data drift over time.

Guardrails LLMOPS

Automated checks and constraints — classifiers, rule sets, or secondary LLM checks — that filter or block unsafe, off-policy, or non-compliant model inputs and outputs. A mandatory control layer for any customer-facing or regulated LLM application, not an optional add-on.

Hallucination Detection LLMOPS

Techniques — grounding checks against retrieved sources, consistency checks, confidence-based flagging — for identifying when a model generates confident but factually incorrect content. A core risk-management requirement for any use case where incorrect output could cause financial, legal, or safety harm.

PII Masking LLMOPS

Detecting and redacting or anonymizing personally identifiable information before it's processed, logged, or sent to a model — on both inputs and outputs. Directly required by privacy regulations (GDPR, CCPA, HIPAA) and a baseline control for any enterprise LLM deployment.

Latency LLMOPS

The time delay between a request and a response, driven by model size, context length, retrieval steps, and chained LLM calls. Often the deciding factor between a synchronous chat UX and an asynchronous, notification-based application design.

Caching LLMOPS

Storing and reusing previous model outputs, retrieval results, or intermediate computations to avoid redundant, costly calls, including prompt-prefix caching for long system prompts. One of the highest-leverage, most underused cost-optimization levers in an LLM platform.

Load Balancing LLMOPS

Distributing inference requests across multiple model instances, providers, or regions to manage throughput and availability. A core reliability pattern for meeting SLAs and avoiding single-vendor dependency risk.

Cost Optimization LLMOPS

Minimizing the compute and API cost of running LLM systems — via model-size routing, caching, prompt-length reduction, batching — while maintaining acceptable quality. LLM spend can scale unpredictably with usage, so cost governance and budget alerts should be built in from day one.

Monitoring LLMOPS

The continuous tracking of system health, quality, and usage metrics in production, spanning both system metrics (latency, error rate) and model-quality metrics (drift, hallucination rate). The operational backbone that turns "we shipped an AI feature" into "we can prove it's working correctly."

Feedback Loop LLMOPS

A mechanism for capturing user or system feedback on model outputs and feeding it back into evaluation, fine-tuning, or prompt improvement. Establishes a continuous-improvement process rather than treating an LLM system as a static, one-time deployment.

A/B Testing LLMOPS

An experimentation method that compares two variants (models, prompts, pipelines) by exposing them to different user segments and measuring outcomes. The gold-standard method for validating that a change actually improves outcomes, not just offline metrics.

Canary Deployment LLMOPS

A rollout strategy that exposes a new model or prompt version to a small fraction of traffic before a full release. Limits the blast radius of a regression by catching quality or safety issues early — a standard risk-mitigation practice given the unpredictability of LLM changes.

Rollback LLMOPS

The ability to quickly revert to a previous known-good model, prompt, or pipeline version when a new release causes problems. Depends on versioning being in place — you can't roll back to a version you didn't track — and is required for any production AI system.

Rate Limiting LLMOPS

Restricting the number of requests a user, application, or system can make to a model within a given time window. Protects downstream systems from overload and defends against both accidental cost overruns and abuse, such as prompt-injection-driven excessive tool use.

Audit LLMOPS

A systematic review of system behavior, decisions, or logs against policy, regulatory, or quality standards, relying on complete tracing to reconstruct what happened. Often a hard regulatory requirement in finance, healthcare, and government — must be designed in, not bolted on.

Governance LLMOPS

The policies, roles, and processes that control how AI systems are approved, deployed, monitored, and retired within an organization. The organizing framework that ties every other concept in this module together into a coherent, accountable AI program.

MODULE 09

Infrastructure

The hardware- and systems-level layer that makes AI workloads run efficiently: containerization and orchestration, specialized inference servers and compilers, GPU/CUDA fundamentals, and the model-efficiency techniques (quantization, KV caching, flash attention, mixture-of-experts) that determine how much a given model costs to serve at scale. This module is where enterprise architecture and platform engineering meet AI most directly.

20 concepts

Docker INFRASTRUCTURE

A containerization platform that packages an application and its dependencies into a portable, isolated unit that runs consistently across environments. It's the standard packaging unit for deploying AI services consistently across dev, staging, and production, and ensures a model's serving environment stays reproducible across machines.

Kubernetes INFRASTRUCTURE

An orchestration platform for deploying, scaling, and managing containerized applications across clusters of machines. It's the de facto standard for enterprise AI platform infrastructure, providing the resource isolation, autoscaling, and fault tolerance needed to run distributed training jobs or many model-serving replicas.

Ray INFRASTRUCTURE

A distributed computing framework for scaling Python-based ML workloads, from data processing to distributed training and serving, across many machines without rewriting core logic. A common backbone for in-house ML platforms that need distributed compute beyond a single vendor's managed service.

vLLM INFRASTRUCTURE

A high-throughput, memory-efficient inference and serving engine for large language models, known for optimized attention and memory management (PagedAttention). A popular choice for self-hosting open-source LLMs, since it significantly reduces GPU cost per request compared to naive serving.

Triton INFRASTRUCTURE

NVIDIA's open-source inference serving platform that runs models from multiple frameworks (PyTorch, TensorFlow, ONNX) with optimized batching behind one serving layer. Consolidates model-serving infrastructure across teams and frameworks, simplifying operational support.

TensorRT INFRASTRUCTURE

NVIDIA's SDK for compiling trained models into highly optimized inference engines via graph optimization, layer fusion, and precision reduction. Meaningfully reduces per-request inference cost, but ties the deployment to NVIDIA-specific tooling and hardware.

CUDA INFRASTRUCTURE

NVIDIA's parallel computing platform and programming model that lets software directly leverage GPU cores for general-purpose computation. Virtually all deep learning frameworks rely on it, and its ecosystem dominance is a key factor behind NVIDIA's position in AI hardware procurement decisions.

GPU INFRASTRUCTURE

Graphics Processing Unit — hardware with thousands of parallel cores well suited to the matrix operations that dominate deep learning. Training and serving large models is impractical on CPUs alone, and GPU procurement/utilization is typically the single largest cost line item in an AI platform budget.

CPU Inference INFRASTRUCTURE

Running trained model inference on standard CPU hardware rather than GPUs, viable for smaller models, batch jobs, or edge deployment. A cost-effective option for lower-volume or latency-tolerant workloads that don't justify GPU infrastructure.

ONNX INFRASTRUCTURE

Open Neural Network Exchange — an open format for representing trained models so they can move between different frameworks and runtimes. Reduces vendor and framework lock-in by standardizing the model interchange format across the deployment pipeline.

Quantization INFRASTRUCTURE

Reducing the numerical precision of a model's weights (e.g., 32-bit to 8-bit or 4-bit) to shrink size and speed up inference, trading a little accuracy for large memory and latency gains. One of the most impactful cost levers for self-hosted models.

KV Cache INFRASTRUCTURE

A memory structure storing previously computed attention key/value tensors during generation, avoiding redundant recomputation for each new token. Its size scales with sequence length and is a major driver of GPU memory requirements — a key factor in per-request serving cost.

Flash Attention INFRASTRUCTURE

An optimized attention algorithm that reduces memory reads/writes by fusing operations, significantly speeding up transformer training and inference. Enables longer context windows than would otherwise be practical on the same hardware, materially affecting the cost of long-context LLM features.

MoE INFRASTRUCTURE

Mixture of Experts — an architecture where only a subset of specialized "expert" sub-networks activates for each input, rather than the full model. Lets models with huge total parameter counts run inference at the cost of a much smaller "active" model — why some very large models are surprisingly cheap to serve.

Distributed Training INFRASTRUCTURE

Splitting model training across multiple GPUs or machines to handle models or datasets too large for a single device, using data, model, or pipeline parallelism. A major driver of training cost and complexity, usually justified only when building or heavily customizing foundation models.

Sharding INFRASTRUCTURE

Splitting a large dataset, model, or index into smaller partitions ("shards") distributed across multiple machines. A standard scalability pattern needed once a single machine can no longer hold the full model, dataset, or vector index.

Replication INFRASTRUCTURE

Running multiple identical copies of a model or service to increase throughput and provide redundancy against failure. The primary mechanism for meeting availability SLAs, since replicated services survive individual node failures.

Autoscaling INFRASTRUCTURE

Automatically adjusting the number of running service instances based on real-time demand, so inference capacity matches traffic without manual intervention. Balances cost efficiency against latency risk — scaling too slowly under GPU-heavy load causes user-facing delays.

Vector Database INFRASTRUCTURE

A database optimized for storing and efficiently searching high-dimensional embedding vectors using approximate nearest-neighbor algorithms. The core storage engine behind semantic search and RAG at production scale, and a new infrastructure category needing its own indexing, scaling, and access-control practices.

API Gateway INFRASTRUCTURE

A layer sitting in front of backend services to manage routing, authentication, rate limiting, and monitoring for API traffic. The natural enforcement point for centralized LLM cost controls, access policy, and usage monitoring across an organization.

MODULE 10

Enterprise AI Platforms & Governance

Where AI meets enterprise IT: the major cloud AI platforms, the orchestration frameworks used to build LLM applications and agents, the emerging interoperability protocols connecting agents and tools, and the governance disciplines — data lineage, compliance, security, identity — that were already core enterprise architecture concerns and now extend directly into AI systems.

20 concepts

Azure AI ENTERPRISE AI

Microsoft's suite of cloud AI services — hosted foundation models, agent-building tools, fine-tuning and evaluation pipelines — integrated with Microsoft's identity, security, and data ecosystem. It's the natural default for organizations already standardized on Microsoft infrastructure, since that integration usually outweighs raw model selection.

AWS Bedrock ENTERPRISE AI

Amazon's managed service for accessing multiple foundation models through one API, with built-in RAG, agent, and fine-tuning tooling. It avoids separate vendor integrations per model and plugs directly into AWS's existing security and networking stack — the default choice for AWS-centric enterprises.

Vertex AI ENTERPRISE AI

Google Cloud's unified ML and generative AI platform, spanning training, hosting, RAG, and agent orchestration. Its key advantage is tight integration between classical ML tooling (AutoML, pipelines) and generative AI — the priority fit for organizations already running on Google Cloud data infrastructure like BigQuery.

LangChain ENTERPRISE AI

An open-source framework of abstractions — chains, retrieval, memory, tool use — for building LLM applications quickly. It accelerates RAG and agent prototyping, but its abstractions often need re-evaluating once a system moves from prototype to production scale.

LangGraph ENTERPRISE AI

A LangChain-based framework that defines agentic workflows as explicit graphs of states and transitions, supporting loops and complex control flow that simple linear chains can't express. The key benefit is auditability — an explicit graph makes agent behavior easier to trace than a freeform agent loop.

Semantic Kernel ENTERPRISE AI

Microsoft's open-source SDK for integrating LLMs into applications, built around a structured, typed plugin and planner architecture. It matters most for teams in the Microsoft/.NET ecosystem, where typed function-calling improves testability over looser prompting frameworks.

CrewAI ENTERPRISE AI

An open-source framework for orchestrating multiple role-based agents ("crews") that collaborate on a shared task with defined responsibilities. Valuable for fast multi-agent prototyping, but production use typically requires layering on separate observability and guardrails the framework doesn't provide out of the box.

AutoGen ENTERPRISE AI

Microsoft's open-source framework for multi-agent conversational systems, where agents — including a human participant — communicate to solve tasks. One of several competing multi-agent frameworks, so the priority decision is community support and fit with existing platform investments, not features alone.

MCP ENTERPRISE AI

Model Context Protocol — an open standard letting LLM applications connect to external tools and data sources through one consistent interface instead of custom integrations. A significant interoperability development: it reduces vendor lock-in and lets a model reuse a growing ecosystem of standardized tool connectors.

A2A ENTERPRISE AI

Agent-to-Agent protocol — an emerging open standard for how independently built agents discover, communicate with, and delegate tasks to one another. Early-stage but strategically important, positioning an organization for a future where agents from different vendors can interoperate without custom point-to-point integration.

Knowledge Graph ENTERPRISE AI

A structured representation of entities and their relationships, letting machines reason over connections rather than just retrieve isolated text. It often overlaps with existing master data management work, making it a natural place to unify AI initiatives with data governance efforts already underway.

Data Governance ENTERPRISE AI

The policies and processes ensuring organizational data is accurate, secure, properly accessed, and compliant with regulation. This is a prerequisite, not an afterthought — weak data governance is one of the most common reasons enterprise AI projects stall before reaching production.

Enterprise Search ENTERPRISE AI

Systems letting employees search across an organization's documents and knowledge sources from one interface, increasingly built on semantic search and RAG rather than legacy keyword indexing. Typically the highest-value, lowest-risk entry point for enterprise LLM adoption, since it's read-only and internally scoped.

Data Lineage ENTERPRISE AI

The tracked history of where data originated, how it was transformed, and where it has been used across systems. Its priority value is traceability — answering "why did the AI say that" by tracing an output back to its originating document or record, which compliance requires.

Compliance ENTERPRISE AI

Adherence to legal, regulatory, and internal policy requirements governing how AI systems handle data and make decisions. Increasingly formalized through AI-specific regulation, such as the EU AI Act, which requires risk classification and documentation by system type — plan for this early, not as a retrofit.

Security ENTERPRISE AI

The practices protecting AI systems and data from unauthorized access, manipulation, or misuse, including AI-specific threats like prompt injection and model extraction. The priority mindset: AI security should extend existing enterprise security architecture with new threat categories, not replace it with a separate program.

Identity ENTERPRISE AI

The systems and standards authenticating and authorizing users, services, and increasingly agents, to control what they can access. The emerging priority issue is agent identity — agents need their own scoped credentials rather than inheriting blanket access under a human user's identity.

Workflow ENTERPRISE AI

A defined business process of ordered steps, systems, and approvals, increasingly automated or augmented with AI components. The highest-value pattern is embedding AI into existing enterprise workflows rather than building AI as a separate silo alongside them.

Event Driven ENTERPRISE AI

An architecture style where components communicate by producing and consuming events, enabling asynchronous, decoupled processing. Well suited to background agentic workflows that don't need an immediate synchronous response, improving both scalability and resilience.

Microservices ENTERPRISE AI

An architecture style structuring an application as small, independently deployable services communicating over APIs. The natural fit for AI platforms, since models, retrieval, and agent logic each have different scaling and update needs that benefit from independent deployment.

APPENDIX

Workflow, Tools & Keywords Matrix

A one-line operational view of every concept in this guide. For each of the 200 concepts: **Workflow** states where it sits in a real pipeline — what comes in, what it does, what comes out. **Tools** lists the libraries, platforms, or products most commonly used to implement it. **Techniques & Keywords** gives the adjacent terms you'll encounter in documentation, vendor material, and job specs.

Tables follow the same module order as the main body. Tool names are indicative of common practice, not endorsements, and the ecosystem moves quickly — treat them as a starting point for evaluation.

Module 01 · AI Foundations

Concept	Workflow (one line)	Tools	Techniques & Keywords
Artificial Intelligence	Business problem → capability-tier selection (rules / ML / LLM / agent) → solution design → risk classification.	Azure AI, AWS Bedrock, Vertex AI, OpenAI/Anthropic APIs	AGI, narrow AI, symbolic AI, capability tiers, AI maturity model
Machine Learning	Labeled data → feature prep → model training → validation → deployment → drift monitoring → retrain.	scikit-learn, MLflow, SageMaker, Databricks	supervised, unsupervised, training set, generalization, model drift
Deep Learning	Raw unstructured data → tensor batching → multi-layer network training on GPU → checkpoint → serve.	PyTorch, TensorFlow, JAX, CUDA, Lightning	representation learning, epochs, layers, GPU cluster, checkpointing
Neural Network	Input tensor → weighted layers + nonlinearity → loss → backprop weight update → trained model artifact.	PyTorch, Keras, ONNX Runtime	weights, biases, layers, forward pass, parameter count
Expert Systems	Domain expert interviews → encode if-then rules → inference engine executes → explainable decision output.	Drools, IBM ODM, CLIPS, Camunda DMN	rule engine, forward/backward chaining, knowledge base, determinism
Knowledge Representation	Raw domain facts → ontology/schema modeling → graph or logic store → queried for machine reasoning.	Neo4j, RDF/OWL, Protégé, Stardog	ontology, triples, taxonomy, semantic layer, schema
Search Algorithms	Define state space → select strategy (BFS/DFS/A*/beam) → explore with heuristic → return optimal path.	OR-Tools, NetworkX, custom solvers	A*, beam search, heuristic, branch and bound, state space
Reasoning	Question → decomposition into intermediate steps → evidence/tool use per step → synthesized conclusion.	Reasoning LLMs, LangGraph, DSPy	chain-of-thought, ReAct, multi-hop, deductive/inductive, self-consistency
Inference	Trained model + new input → forward pass on serving hardware → output + confidence → downstream consumer.	vLLM, Triton, TorchServe, Bedrock	serving, batch vs. real-time, throughput, TTFT, cost per call
Optimization	Define objective + constraints → choose optimizer → iterate toward minimum → converged parameters.	Optuna, SciPy, Adam/SGD, OR-Tools	objective function, constraints, convergence, local minimum, search
Planning	Goal + current state → decompose into ordered actions → validate feasibility → hand plan to executor.	LangGraph, CrewAI, PDDL planners	task decomposition, goal state, action sequence, replanning
Learning	Select paradigm by available signal → expose model to data/feedback → update behavior → evaluate improvement.	PyTorch, RLlib, scikit-learn, TRL	supervised, self-supervised, reinforcement, transfer, online learning
Prediction	New input → trained model scoring → label/value + probability → threshold check → business action.	SageMaker Endpoints, Vertex Prediction, FastAPI	scoring, confidence, point estimate, prediction interval, ground truth
Classification	Labeled examples → train classifier → predict class + probability → threshold tuning → fairness audit.	XGBoost, scikit-learn, BERT, Vertex AutoML	binary/multiclass, precision, recall, F1, ROC-AUC, class imbalance
Regression	Historical numeric outcomes → fit model → predict continuous value + error bounds → feed planning systems.	scikit-learn, XGBoost, Prophet, statsmodels	RMSE, MAE, R ² , residuals, forecasting, confidence interval

Clustering	Unlabeled data → distance/density grouping → cluster profiling → segment naming → periodic revalidation.	scikit-learn , HDBSCAN , Spark MLlib	centroid, silhouette score, elbow method, segmentation, unsupervised
Anomaly Detection	Baseline normal behavior → score incoming events → threshold breach → alert → analyst triage feedback.	Isolation Forest , PyOD , Splunk , Elastic ML	outlier, novelty detection, false positive rate, drift, thresholding
Recommendation	User + item interaction log → candidate generation → ranking → serve top-N → A/B measure engagement.	TensorFlow Recommenders , Feast , Redis , Merlin	collaborative filtering, cold start, CTR, implicit feedback, ranking
Decision Support	Model output + context + uncertainty → presented in UI → human judgment → logged decision → feedback loop.	Power BI , Tableau , Streamlit , internal apps	augmented intelligence, explainability, uncertainty display, HITL
Automation	Process mining → identify repetitive high-volume task → automate with rules/ML/agent → set oversight tier.	UiPath , Power Automate , Airflow , Temporal	RPA, straight-through processing, exception handling, oversight tier

Module 02 · Mathematics

Concept	Workflow (one line)	Tools	Techniques & Keywords
Linear Algebra	Data as vectors/matrices → layer-wise matrix multiplication → transformed representation → GPU-accelerated throughput.	NumPy, PyTorch, BLAS/cuBLAS	dot product, matmul, rank, transpose, linear transformation
Calculus	Loss surface → partial derivatives via chain rule → gradient signal → weight update direction.	Autograd (PyTorch), JAX, SymPy	chain rule, partial derivative, Jacobian, Hessian, differentiation
Probability	Uncertain event → distribution modeling → likelihood scoring → confidence threshold → automation decision.	SciPy, PyMC, TensorFlow Probability	distribution, likelihood, prior/posterior, softmax, sampling
Statistics	Hypothesis → sample design → test execution → significance check → ship/no-ship decision.	statsmodels, R, SciPy, Optimizely	p-value, confidence interval, power analysis, sampling bias, significance
Vectors	Raw text/image/user → encoder → dense vector → similarity computation → ranked matches.	NumPy, sentence-transformers, FAISS	cosine similarity, Euclidean distance, dimensionality, norm, embedding
Matrices	Weights stored as matrices → batched multiplication per layer → activations → memory footprint drives cost.	NumPy, cuBLAS, PyTorch	matmul, shape, sparse matrix, parameter count, memory footprint
Eigenvalues	High-dim matrix → eigen-decomposition → principal directions → dimensionality reduction → storage savings.	NumPy linalg, scikit-learn PCA, SciPy	eigenvector, SVD, spectral decomposition, variance explained
Gradients	Loss → backprop computes partials → gradient vector → optimizer step → clipping if unstable.	PyTorch autograd, JAX, TensorBoard	vanishing/exploding gradient, gradient clipping, learning rate, descent
Bayes Theorem	Prior belief + new evidence → likelihood weighting → posterior update → revised confidence.	PyMC, Stan, scikit-learn NB	prior, posterior, likelihood, conditional probability, Bayesian update
Entropy	Predicted distribution vs. true label → cross-entropy loss → training signal; high entropy → low-confidence flag.	PyTorch losses, TensorFlow, scikit-learn	cross-entropy, KL divergence, information gain, perplexity, uncertainty
Optimization	Objective + constraints → solver search → feasible optimum → applied to training or resource allocation.	Optuna, OR-Tools, CVXPY, SciPy	constrained optimization, Lagrangian, global vs. local optimum, solver
Convexity	Inspect loss surface shape → convex means reliable single optimum; non-convex means run-to-run variance.	CVXPY, SciPy optimize	global minimum, saddle point, non-convex, loss landscape, reproducibility
Markov Chains	Current state → transition probability matrix → next state → iterate to steady-state distribution.	PyMC (MCMC), NetworkX, hmmlearn	transition matrix, memoryless, MDP, steady state, HMM
Graph Theory	Entities as nodes + relations as edges → graph store → traversal/GNN → relationship-aware output.	Neo4j, NetworkX, PyTorch Geometric, DGL	nodes, edges, traversal, centrality, GNN, shortest path
Set Theory	Define document/user sets → intersection/union operations → filtered candidate set → access-scoped results.	SQL, Pandas, Elasticsearch filters	union, intersection, membership, deduplication, cardinality, RBAC
Combinatorics	Enumerate configuration space → recognize explosion → switch to sampling/heuristic search → bounded run.	Optuna, Ray Tune, itertools	permutations, search space, combinatorial explosion, sampling, pruning

Differential Equations	Forward noising process defined → model learns reverse → iterative denoising → generated sample.	Diffusers , PyTorch , torchdiffeq	SDE/ODE, denoising, noise schedule, continuous-time, Neural ODE
Numerical Methods	Exact solution infeasible → approximate iteratively → apply stability guards → choose precision level.	SciPy , NumPy , AMP (mixed precision)	FP32/FP16/BF16/INT8, numerical stability, gradient clipping, rounding error
Tensor Algebra	Batch data reshaped into tensors → broadcast operations across dims → GPU kernels → tuned throughput.	PyTorch , TensorFlow , JAX , einsum	rank, shape, broadcasting, einsum, batch dimension, contraction
Random Variables	Define distribution → sample at inference (temperature/top-p) → variable output → QA accounts for variance.	NumPy random , PyTorch , seed control	sampling, temperature, top-k/top-p, stochasticity, seed, expectation

Module 03 · Classical Machine Learning

Concept	Workflow (one line)	Tools	Techniques & Keywords
Decision Tree	Tabular data → recursive threshold splits → pruned tree → traceable rule path per prediction.	scikit-learn, R rpart, Spark MLlib	Gini impurity, entropy split, pruning, max depth, interpretability
Random Forest	Bootstrap samples → train many trees on feature subsets → average/vote → stable low-risk prediction.	scikit-learn, H2O, Spark MLlib	bagging, out-of-bag error, feature importance, ensemble, variance reduction
XGBoost	Tabular features → sequential boosted trees correcting residuals → tuned model → SHAP explanation → deploy.	XGBoost, SHAP, MLflow, SageMaker	gradient boosting, residuals, regularization, early stopping, SHAP values
LightGBM	Large tabular set → histogram binning → leaf-wise tree growth → fast retrain cycle in production pipeline.	LightGBM, Dask, Databricks	histogram binning, leaf-wise growth, GOSS, training speed, memory efficiency
CatBoost	Raw categorical business data → ordered target encoding internally → boosted trees → minimal preprocessing pipeline.	CatBoost, Yandex tooling, MLflow	ordered boosting, target encoding, categorical features, CRM/ERP data
SVM	Feature vectors → find max-margin hyperplane (kernel if non-linear) → compact classifier → fast edge inference.	scikit-learn, LIBSVM, ThunderSVM	kernel trick, RBF, margin, support vectors, hinge loss
KNN	Store all training points → at query compute distances → take k nearest → majority vote label.	scikit-learn, FAISS, Annoy, ScaNN	distance metric, k selection, ANN index, lazy learning, curse of dimensionality
Naive Bayes	Token counts → class-conditional probabilities → Bayes combination → fast text label.	scikit-learn, NLTK, Spark MLlib	conditional independence, Laplace smoothing, TF-IDF, spam filtering
Logistic Regression	Features → weighted sum → sigmoid → probability → threshold → coefficients explained to auditors.	scikit-learn, statsmodels, SAS	sigmoid, odds ratio, coefficients, regularization (L1/L2), interpretability
Linear Regression	Historical numeric data → least-squares fit → coefficients → continuous forecast baseline.	scikit-learn, statsmodels, Excel	OLS, coefficients, multicollinearity, R ² , residual analysis
PCA	High-dim features → covariance eigen-decomposition → top-k components → compressed features into model.	scikit-learn, NumPy, UMAP (alt)	variance explained, components, SVD, whitening, dimensionality reduction
KMeans	Choose K → assign points to nearest centroid → recompute centroids → converge → profile segments.	scikit-learn, Spark MLlib, FAISS	centroid, inertia, elbow method, k-means++, silhouette score
DBSCAN	Set eps/minPts → expand dense regions into clusters → label sparse points as noise → route outliers to review.	scikit-learn, HDBSCAN	density reachability, eps, minPts, noise points, arbitrary cluster shape
Hierarchical Clustering	Compute pairwise distances → iteratively merge/split → dendrogram → cut at chosen granularity → taxonomy.	SciPy, scikit-learn, seaborn clustermap	dendrogram, linkage (ward/average), agglomerative, divisive, cut height
Feature Engineering	Raw tables → derive/aggregate/encode features → feature store → consistent train & serve inputs.	Feast, Tecton, Pandas, dbt, Spark	feature store, encoding, binning, scaling, train-serve skew, lineage

Feature Selection	Candidate features → importance/correlation filtering → reduced set → simpler, better-governed model.	scikit-learn , SHAP , Boruta , mlxtend	filter/wrapper/embedded, RFE, mutual information, multicollinearity
Cross Validation	Split into k folds → train/test rotate → average metrics → reliable estimate before go-live sign-off.	scikit-learn , MLflow , Optuna	k-fold, stratified, leave-one-out, time-series split, data leakage
Hyperparameter Tuning	Define search space + budget cap → run search trials → select best config → retrain final model.	Optuna , Ray Tune , Hyperopt , SageMaker AMT	grid/random search, Bayesian optimization, early stopping, compute budget
Bias	Evaluate errors overall and per subgroup → diagnose underfit vs. unfair treatment → mitigate → formal review.	Fairlearn , AIF360 , What-If Tool , SHAP	underfitting, disparate impact, protected attributes, fairness metrics
Variance	Compare train vs. validation gap → detect overfitting → regularize/ensemble/add data → stable production behavior.	scikit-learn , Evidently AI , MLflow	overfitting, regularization, bias-variance trade-off, ensembling, drift

Module 04 · Deep Learning

Concept	Workflow (one line)	Tools	Techniques & Keywords
CNN	Image batch → convolution + pooling layers → feature maps → classifier head → vision prediction.	PyTorch, TensorFlow, OpenCV, timm	kernel, stride, padding, pooling, feature map, translation invariance
RNN	Sequence input → step-by-step hidden state update → final/per-step output → sequence prediction.	PyTorch, Keras, TensorFlow	hidden state, unrolling, BPTT, sequence modeling, vanishing gradient
LSTM	Long sequence → input/forget/output gates manage cell state → retained long-range signal → forecast.	PyTorch, Keras, Darts, GluonTS	cell state, forget gate, long-range dependency, time series, gating
GRU	Sequence → reset/update gates (fewer params than LSTM) → faster training → edge/on-device deployment.	PyTorch, Keras, TFLite, ONNX	update gate, reset gate, lightweight, edge inference, parameter efficiency
Transformer	Tokenized sequence + positional info → stacked self-attention blocks (parallel) → contextual output tokens.	Hugging Face Transformers, PyTorch, vLLM	encoder/decoder, multi-head attention, context window, scaling laws
Attention	Query/key/value projection → relevance scores → weighted value sum → context-aware representation.	PyTorch, FlashAttention, xFormers	Q/K/V, softmax scores, cross-attention, multi-head, quadratic cost
Self Attention	Sequence attends to itself → every token weighted against every other → contextual embedding per token.	Hugging Face, FlashAttention, PyTorch SDPA	$O(n^2)$ scaling, causal masking, context length, token interaction
Positional Encoding	Token embeddings + position signal injected → order-aware input → supports long-context extrapolation.	RoPE, ALiBi, Hugging Face configs	sinusoidal, learned, rotary (RoPE), ALiBi, context extension, NTK scaling
Batch Normalization	Layer activations → normalize across mini-batch → scale/shift → faster, more stable convergence.	PyTorch, TensorFlow, Keras	LayerNorm, RMSNorm, internal covariate shift, training stability
Dropout	During training randomly zero a fraction of units → prevent co-adaptation → disable at inference.	PyTorch, TensorFlow, Keras	regularization, dropout rate, overfitting, generalization, inference mode
Residual Network	Layer output + skip-connected input → unimpeded gradient flow → trainable very deep stacks.	PyTorch (ResNet), timm, Hugging Face	skip connection, identity mapping, depth, gradient flow, ResNet blocks
Autoencoder	Input → encoder compresses to bottleneck → decoder reconstructs → reconstruction error flags anomalies.	PyTorch, Keras, PyOD	latent space, bottleneck, VAE, denoising, reconstruction error
GAN	Generator creates sample → discriminator judges real/fake → adversarial updates → realistic synthetic data.	PyTorch, StyleGAN, SDV, TensorFlow GAN	generator, discriminator, mode collapse, adversarial loss, synthetic data
Diffusion	Random noise → iterative learned denoising steps conditioned on prompt → generated image/video.	Diffusers, Stable Diffusion, ComfyUI	denoising steps, noise schedule, latent diffusion, guidance scale, U-Net
Vision Transformer	Image split into patches → patches embedded as tokens → transformer blocks → vision/multimodal output.	timm, Hugging Face ViT, CLIP, SigLIP	patch embedding, CLS token, ViT, CLIP, multimodal, patch size
Activation Function	Linear layer output → nonlinear transform → enables complex function approximation in next layer.	PyTorch, TensorFlow (built-in)	ReLU, GELU, SiLU/Swish, sigmoid, tanh, dying ReLU, nonlinearity

Backpropagation	Forward pass → loss → reverse chain-rule gradient computation → per-weight update signal.	PyTorch autograd, JAX, TensorFlow GradientTape	chain rule, computational graph, autograd, reverse-mode differentiation
Gradient Descent	Gradient → step opposite direction scaled by learning rate → repeat over batches → converged weights.	PyTorch optim, TensorFlow, DeepSpeed	SGD, mini-batch, momentum, learning rate schedule, convergence
Adam Optimizer	Track gradient moment estimates → adapt per-parameter learning rate → robust convergence with little tuning.	PyTorch AdamW, DeepSpeed, Optax	AdamW, momentum, weight decay, adaptive learning rate, beta1/beta2
Loss Function	Prediction vs. target → scalar error → gradient source; choice must encode the real business objective.	PyTorch losses, TensorFlow, custom losses	cross-entropy, MSE, contrastive, triplet, objective alignment, custom loss

Module 05 · Natural Language Processing

Concept	Workflow (one line)	Tools	Techniques & Keywords
Tokenization	Raw text → split into subword tokens → map to IDs → model input; token count drives API cost.	tiktoken, SentencePiece, Hugging Face Tokenizers, BPE	BPE, WordPiece, subword, vocabulary size, token limit, cost per token
Stemming	Text → strip suffixes by rule → crude root form → keyword index normalization (legacy pipelines).	NLTK, Snowball, Elasticsearch analyzers	Porter stemmer, suffix stripping, over-stemming, index normalization
Lemmatization	Text + POS context → dictionary lookup → true base form → precise normalized text for compliance search.	spaCy, NLTK WordNet, Stanza	lemma, morphological analysis, dictionary-based, POS-aware normalization
NER	Document → entity span detection + type classification → structured entities → redaction or DB population.	spaCy, Presidio, AWS Comprehend, GLiNER	entity span, BIO tagging, PII detection, redaction, entity linking
POS Tagging	Sentence → assign grammatical tag per token → input to parser or rule-based checker.	spaCy, NLTK, Stanza, CoreNLP	Penn Treebank tags, sequence labeling, grammatical role, syntactic feature
Dependency Parsing	Sentence → build head-dependent relation tree → extract subject-verb-object triples → structured facts.	spaCy, Stanza, CoreNLP	dependency tree, head/dependent, SVO extraction, relation extraction
Word2Vec	Corpus → train skip-gram/CBOW on context windows → static word vectors → similarity lookups.	Gensim, TensorFlow, Spark MLlib	skip-gram, CBOW, negative sampling, static embedding, word analogy
FastText	Corpus → character n-gram embeddings → vectors robust to typos/rare words → fast classifier.	fastText (Meta), Gensim	character n-grams, subword embedding, OOV handling, morphology-rich languages
BERT	Text → bidirectional encoder → contextual embeddings → task head (classify/NER) → fine-tune on domain data.	Hugging Face, PyTorch, ONNX Runtime	masked language modeling, encoder-only, CLS token, fine-tuning, DistilBERT
RoBERTa	Same as BERT with improved pretraining recipe → higher-accuracy encoder → drop-in upgrade for understanding tasks.	Hugging Face, fairseq, PyTorch	dynamic masking, larger batches, pretraining recipe, encoder upgrade
T5	Any NLP task reframed as text-in/text-out → encoder-decoder → unified multi-task model.	Hugging Face, Flan-T5, JAX/T5X	text-to-text, encoder-decoder, seq2seq, task prefix, Flan tuning
GPT	Prompt → decoder-only next-token prediction → streamed generation → app response or agent action.	OpenAI, Anthropic, Llama, Mistral, Bedrock	autoregressive, decoder-only, next-token prediction, context window, streaming
Sentence Embeddings	Passage → embedding model → dense vector → vector index; same model must be used for query and index.	sentence-transformers, OpenAI/Cohere embeddings, BGE, E5	bi-encoder, cosine similarity, embedding dimension, MTEB benchmark
Prompt Engineering	Task goal → draft instruction + examples + format spec → test against eval set → version and deploy.	LangSmith, PromptLayer, DSPy, Langfuse	few-shot, zero-shot, system prompt, chain-of-thought, output schema, templating
Instruction Tuning	Base model + (instruction, response) dataset → supervised fine-tune → instruction-following assistant model.	TRL, Axolotl, Hugging Face, Unsloth	SFT, instruction dataset, base vs. instruct model, alignment, chat template
RLHF	Collect human preference pairs → train reward model → RL policy optimization → aligned model behavior.	TRL, DPO implementations, OpenRLHF	reward model, PPO, DPO, preference pairs, alignment, RLAI, constitutional AI

LoRA	Freeze base weights → train small low-rank adapter matrices → merge or serve adapter per domain.	PEFT (Hugging Face), Axolotl, vLLM LoRA	low-rank, rank/alpha, adapter, frozen base, multi-tenant adapters
QLoRA	Quantize base model to 4-bit → attach LoRA adapters → fine-tune on a single GPU → serve.	bitsandbytes, PEFT, Unsloth, Axolotl	4-bit NF4, double quantization, single-GPU fine-tuning, memory efficiency
PEFT	Choose adapter method → tune small parameter subset → evaluate → deploy without full model copies.	Hugging Face PEFT, Adapters, LLaMA-Factory	LoRA, prefix tuning, prompt tuning, adapters, catastrophic forgetting
Fine Tuning	Curate domain dataset → continue training pretrained model → evaluate vs. prompting baseline → version and monitor.	Hugging Face, OpenAI FT API, Vertex, Bedrock	domain adaptation, catastrophic forgetting, dataset curation, eval baseline

Module 06 · Retrieval-Augmented Generation

Concept	Workflow (one line)	Tools	Techniques & Keywords
Semantic Search	Query → embed → ANN search over vector index → ranked passages by meaning → into LLM context.	Pinecone, Weaviate, Qdrant, pgvector, Elastic	cosine similarity, ANN, recall@k, embedding model, top-k retrieval
BM25	Query terms → inverted index lookup → TF-IDF-style scoring → keyword-matched documents.	Elasticsearch, OpenSearch, Lucene, rank_bm25	TF-IDF, inverted index, term frequency, IDF, exact match, lexical search
Hybrid Search	Run BM25 + dense retrieval in parallel → fuse scores (RRF/weighted) → merged candidate set → rerank.	Elasticsearch hybrid, Weaviate, Vespa, Qdrant	reciprocal rank fusion (RRF), score normalization, alpha weighting, fusion
Dense Retrieval	Chunks → embed at index time → query embedded → nearest-neighbor match → re-embed when model changes.	FAISS, Milvus, Pinecone, sentence-transformers	bi-encoder, HNSW, IVF, embedding drift, reindexing, vector dimension
Sparse Retrieval	Documents → term-based inverted index → exact-term query match → precise keyword results.	Elasticsearch, Solr, SPLADE, Lucene	inverted index, term weighting, SPLADE (learned sparse), lexical precision
ColBERT	Token-level embeddings stored per doc → late-interaction MaxSim scoring at query → high-precision ranking.	RAGatouille, ColBERT-v2, Vespa	late interaction, MaxSim, token-level embedding, multi-vector index, PLAID
Parent Child Retrieval	Index small child chunks → match on child → return larger parent doc → richer LLM context.	LangChain ParentDocumentRetriever, LlamaIndex	chunk size, small-to-big, context window, chunk overlap, hierarchical chunking
Multi Query Retrieval	User query → LLM generates N paraphrases → retrieve for each → deduplicate and merge results.	LangChain MultiQueryRetriever, LlamaIndex	query rewriting, paraphrase generation, recall improvement, result fusion
Multi Vector Retrieval	Per document create several embeddings (chunk, summary, hypothetical Q) → index all → broader match coverage.	LangChain MultiVectorRetriever, LlamaIndex	summary embedding, hypothetical questions, index size, coverage, doc store
Context Compression	Retrieved chunks → relevance filter/summarize → condensed context → shorter, cheaper, cleaner prompt.	LLMLingua, LangChain ContextualCompressionRetriever	prompt compression, extractive filtering, lost-in-the-middle, token reduction
HyDE	Query → LLM writes hypothetical answer → embed that answer → retrieve documents similar to it.	LangChain HyDE, LlamaIndex	hypothetical document, query-document mismatch, zero-shot dense retrieval
Query Expansion	Query → add synonyms/related terms/context → broadened query → higher-recall retrieval.	Elasticsearch synonyms, LangChain, LLM rewriters	synonym expansion, pseudo-relevance feedback, recall, query rewriting
Metadata Filtering	Attach metadata at ingest → apply user/date/dept filters → scoped candidate set → semantic rank within it.	Pinecone/Qdrant/Weaviate filters, pgvector + SQL	pre/post-filtering, ACL enforcement, tenant isolation, faceting, payload
Recursive Retrieval	Retrieve → assess sufficiency → generate follow-up query → retrieve again → stop at iteration limit.	LlamaIndex, LangGraph, DSPy	multi-hop, iterative refinement, stopping criteria, hop limit, sub-questions
Re Ranking	Top-50 fast candidates → cross-encoder scores each query-doc pair → reordered top-5 → generation.	Cohere Rerank, bge-reranker, Jina, cross-encoders	cross-encoder, two-stage retrieval, NDCG, MRR, precision@k, latency budget

Knowledge Graph RAG	Extract entities/relations → build graph → traverse for connected facts → combine with text chunks in context.	Neo4j, Microsoft GraphRAG, LlamaIndex KG	entity extraction, triples, community detection, multi-hop reasoning, Cypher
Self RAG	Retrieve → generate → model critiques its own grounding → retrieve more or revise → verified answer.	LangGraph, LlamaIndex, custom critique loops	self-reflection, reflection tokens, groundedness check, hallucination reduction
CRAG	Retrieve → grade document relevance → if poor, rewrite query or fall back to web search → then generate.	LangGraph, Tavily/Serper (fallback), LlamaIndex	retrieval grading, corrective fallback, web search tool, knowledge gap
Adaptive RAG	Classify query complexity → route to no-retrieval / single-pass / multi-step → execute cheapest sufficient path.	LangGraph, semantic router, LlamaIndex	query classification, routing, cost per query, complexity tiering, no-retrieval
Agentic RAG	Agent plans → selects sources/tools dynamically → iterates retrieval + reasoning → synthesized cited answer.	LangGraph, LlamaIndex agents, CrewAI, MCP	tool selection, dynamic planning, multi-source, observability, cost ceiling

Module 07 · Agentic AI

Concept	Workflow (one line)	Tools	Techniques & Keywords
Planner	Goal → LLM reasons step-by-step → structured ordered plan → inspected/approved → passed to executor.	LangGraph, CrewAI, AutoGen, Semantic Kernel	task decomposition, plan schema, replanning, approval gate, DAG
Executor	Plan step → resolve tool + arguments → invoke API/function → capture result → log side effects.	MCP servers, LangChain tools, function calling	tool invocation, side effects, idempotency, least privilege, audit log
Supervisor	Receive task → dispatch to worker agents → evaluate returns → decide next hop → assemble final output.	LangGraph supervisor, AutoGen, CrewAI	orchestration, worker agents, handoff, routing logic, centralized control
Reflection	Draft output → self-critique against criteria → identify flaws → revise → finalize improved answer.	LangGraph, Reflexion patterns, DSPy	self-critique, revision loop, Reflexion, iteration cap, quality gain vs. cost
Memory	Interaction → write to short-term context or long-term store → retrieve relevant memories on later turns.	Mem0, Zep, LangGraph checkpoint, Redis, vector DB	episodic/semantic memory, summarization buffer, retention policy, PII, TTL
Tool Calling	Model emits structured tool call → runtime validates schema → executes → returns result into context.	OpenAI/Anthropic function calling, MCP, LangChain	JSON schema, function calling, parallel tools, permission scope, tool registry
Multi Agent	Decompose task by role → specialized agents work in parallel/sequence → results merged → traced end-to-end.	CrewAI, AutoGen, LangGraph, A2A	role specialization, agent handoff, message passing, emergent failure modes
Coordinator	Maintain shared state → sequence agent hand-offs → resolve conflicts → enforce cross-agent policy.	LangGraph state, Temporal, Redis, AutoGen	shared state, synchronization, conflict resolution, orchestration layer
Router	Incoming request → fast intent/complexity classification → dispatch to cheapest capable model or agent.	Semantic Router, LangGraph, RouteLLM, LiteLLM	intent classification, model tiering, cost routing, fallback chain, guard model
Critic	Receive another agent's output → score against rubric → approve or return with correction notes.	LangGraph, AutoGen critic agents, Guardrails AI	rubric, quality gate, actor-critic, error propagation, automated review
Judge	Candidate outputs + rubric → LLM scores/compares → aggregated eval metric → human spot-check calibration.	Ragas, DeepEval, LangSmith, Braintrust, promptfoo	LLM-as-judge, pairwise comparison, rubric, position bias, calibration
Swarm	Broadcast task → many lightweight agents explore in parallel → collect diverse candidates → select best.	OpenAI Swarm, Ray, custom async orchestration	parallel exploration, decentralized, emergent behavior, cost unpredictability
Workflow	Predefined step graph → deterministic execution with LLM calls at fixed nodes → predictable output.	Temporal, Airflow, Prefect, LangGraph, n8n	DAG, deterministic steps, retry policy, state machine, orchestration
Human in the Loop	Agent reaches checkpoint → pause and surface for review → human approves/edits → execution resumes → feedback captured.	LangGraph interrupts, Slack/Teams approvals, Temporal signals	approval gate, interrupt/resume, escalation, review queue, feedback data
Autonomous Agent	Goal → plan → act → observe → self-correct loop until done or limit hit → report result.	LangGraph, AutoGPT-style, Claude Agent SDK	agent loop, cost ceiling, timeout, kill switch, max iterations, guardrails

Reasoning Agent	Observe → think (visible reasoning) → act via tool → observe result → repeat until answer.	ReAct in LangChain, reasoning models, LangGraph	ReAct, thought-action-observation, scratchpad, reasoning trace, tool loop
Planning Agent	Goal → produce and refine candidate plans → validate feasibility → human approval → hand to executors.	LangGraph plan-and-execute, CrewAI, Semantic Kernel	plan-and-execute, plan validation, refinement, separation of concerns
Browser Agent	Navigate URL → read DOM/screenshot → decide action → click/type → verify state → repeat within allow-list.	Playwright, Browserbase, Claude in Chrome, browser-use	DOM parsing, computer use, domain allow-list, prompt injection risk, CAPTCHA
Code Agent	Task → generate code → run in sandbox → read errors → iterate → PR raised for human review.	Claude Code, E2B, Docker sandbox, GitHub Actions	sandboxing, test-driven loop, dependency allow-list, code review gate, REPL
Research Agent	Question → plan sub-queries → search & read sources → synthesize with citations → verifiable report.	Tavily, Perplexity API, LangGraph, GPT Researcher	sub-question decomposition, source citation, synthesis, traceability, coverage

Module 08 · LLMOps

Concept	Workflow (one line)	Tools	Techniques & Keywords
Prompt Versioning	Edit prompt → commit new version → run against eval set → compare to baseline → promote or revert.	LangSmith, Langfuse, PromptLayer, Git	prompt registry, diff, A/B compare, rollback, change management, templating
Model Registry	Trained model + metadata → register with version → stage transitions → deploy → auditable lineage record.	MLflow, SageMaker Registry, Vertex, W&B	model lineage, staging/production tags, artifact store, reproducibility
Tracing	Request → capture spans for each prompt/tool/retrieval → store trace tree → inspect on failure.	LangSmith, Langfuse, Arize Phoenix, OpenTelemetry	span, trace tree, OTel GenAI semantics, latency breakdown, replay
Observability	Instrument logs + metrics + traces → dashboards → alerts on quality/cost/latency → investigate regressions.	Langfuse, Arize, Datadog LLM, Grafana, Helicone	metrics/logs/traces, drift detection, dashboards, SLO, alerting
Evaluation	Curate golden dataset → run automated + LLM-judge + human scoring → track scores per release, continuously.	Ragas, DeepEval, LangSmith, promptfoo, Braintrust	golden dataset, groundedness, faithfulness, answer relevance, regression suite
Guardrails	Input screened → model call → output screened → block/redact/rewrite on policy violation → log incident.	Guardrails AI, NeMo Guardrails, Llama Guard, Bedrock Guardrails	input/output filtering, jailbreak detection, prompt injection, topic restriction
Hallucination Detection	Answer + retrieved sources → groundedness/entailment scoring → flag unsupported claims → block or annotate.	Ragas, TruLens, Vectara HHEM, Galileo	groundedness, faithfulness, citation verification, NLI entailment, self-consistency
PII Masking	Detect entities in input → redact/tokenize before model call → optionally re-identify → scrub logs too.	Microsoft Presidio, AWS Comprehend PII, Skyflow	de-identification, tokenization, reversible masking, GDPR/HIPAA, log scrubbing
Latency	Measure TTFT and total time per stage → find bottleneck (retrieval/model/chain) → optimize or go async.	Langfuse, Datadog, k6, Locust	TTFT, tokens/sec, p95/p99, streaming, async UX, cold start
Caching	Hash prompt/query → check cache → hit returns instantly, miss calls model and stores result → TTL expiry.	Redis, GPTCache, prompt caching APIs, Semantic Cache	exact vs. semantic cache, prompt prefix caching, TTL, hit rate, invalidation
Load Balancing	Request → gateway selects healthy provider/replica → failover on error → even utilization across pool.	LiteLLM, Portkey, Kong, NGINX, Envoy	round-robin, health check, failover, multi-provider, quota distribution
Cost Optimization	Attribute spend per feature/user → apply routing, caching, prompt trimming, batching → track cost per request.	Helicone, LiteLLM, Portkey, cloud cost tools	cost per request, model tiering, batch API, prompt trimming, budget alerts
Monitoring	Stream production metrics → dashboard system + quality signals → alert on threshold breach → triage.	Grafana, Datadog, Evidently, Arize, Prometheus	drift, error rate, quality score, user feedback rate, alert thresholds, SLO
Feedback Loop	Capture thumbs/edits/outcomes → store with trace → triage into eval set or training data → improve → redeploy.	LangSmith, Langfuse, Label Studio, Argilla	implicit/explicit feedback, annotation queue, data flywheel, RLHF input
A/B Testing	Split traffic between variants → hold constant → collect outcome metrics → significance test → roll winner out.	LaunchDarkly, Optimizely, Statsig, GrowthBook	traffic split, control/treatment, statistical significance, guardrail metrics

Canary Deployment	Release to 1-5% traffic → watch quality/error/cost metrics → auto-rollback on regression → ramp gradually.	Argo Rollouts, Flagger, LaunchDarkly, Istio	progressive rollout, blast radius, auto-rollback trigger, traffic ramp
Rollback	Detect regression → select last known-good version from registry → repoint traffic → verify recovery.	MLflow, Argo, Git revert, feature flags	known-good version, MTTR, pinning, blue-green, instant revert
Rate Limiting	Identify caller → check quota window → allow or throttle/429 → protect cost and downstream systems.	Kong, Redis, LiteLLM, API Gateway, Envoy	token bucket, per-tenant quota, 429 backoff, burst limit, abuse prevention
Audit	Retain complete traces/logs → reconstruct any decision on request → produce evidence for reviewers.	Langfuse, Splunk, CloudTrail, immutable log stores	immutable logs, retention policy, decision reconstruction, evidence, WORM
Governance	Use case intake → risk classification → review/approval → controlled deployment → periodic re-review → retirement.	Credo AI, IBM watsonx.governance, ServiceNow, Collibra	AI risk register, model cards, EU AI Act tiers, approval workflow, NIST AI RMF

Module 09 · Infrastructure

Concept	Workflow (one line)	Tools	Techniques & Keywords
Docker	Code + deps + drivers → Dockerfile build → image pushed to registry → identical run in any environment.	Docker, BuildKit, ECR/ACR/GCR, Podman	image layers, Dockerfile, container registry, reproducibility, base image
Kubernetes	Container image → deployment manifest → scheduler places pods on GPU nodes → autoscale → self-heal.	Kubernetes, Helm, KServe, Kubeflow, EKS/AKS/GKE	pods, node pools, HPA, GPU scheduling, resource limits, operators
Ray	Python workload → distributed tasks/actors across cluster → parallel training or serving → aggregated result.	Ray Core, Ray Train, Ray Serve, Ray Tune, Anyscale	actors, task parallelism, distributed scheduler, cluster autoscaling
vLLM	Load open-weight model → continuous batching + PagedAttention → high-throughput OpenAI-compatible endpoint.	vLLM, SGLang, TGI, Ray Serve	PagedAttention, continuous batching, throughput, tensor parallel, OpenAI API
Triton	Models from multiple frameworks → model repository → Triton server with dynamic batching → unified endpoint.	NVIDIA Triton, TensorRT-LLM, KServe	dynamic batching, model repository, multi-framework, concurrent execution
TensorRT	Trained model → graph optimization + layer fusion + precision calibration → compiled engine → fast GPU inference.	TensorRT, TensorRT-LLM, ONNX, Torch-TensorRT	layer fusion, kernel auto-tuning, INT8 calibration, engine build, NVIDIA lock-in
CUDA	Framework op → CUDA kernel dispatch → parallel execution across GPU cores → result back to host.	CUDA Toolkit, cuDNN, cuBLAS, Triton (OpenAI)	kernels, SMs, warps, device memory, compute capability, ecosystem lock-in
GPU	Size model + batch against VRAM → provision instances → schedule workloads → monitor utilization → optimize spend.	NVIDIA H100/A100/L40S, DCGM, cloud GPU instances	VRAM, HBM, NVLink, utilization, MIG, spot instances, capacity planning
CPU Inference	Small/quantized model → CPU-optimized runtime → adequate latency at low cost for modest volume.	ONNX Runtime, llama.cpp, OpenVINO, GGUF	AVX/AMX, GGUF, quantized CPU inference, edge deployment, cost per inference
ONNX	Train in PyTorch → export to ONNX graph → optimize → run on any compatible runtime/hardware.	ONNX, ONNX Runtime, Netron, Optimum	graph export, opset version, interoperability, runtime portability, operator support
Quantization	FP16 weights → calibrate → convert to INT8/INT4 → validate quality delta → deploy smaller, faster model.	bitsandbytes, GPTQ, AWQ, llama.cpp, TensorRT	PTQ vs. QAT, INT8/INT4, GPTQ, AWQ, perplexity delta, memory reduction
KV Cache	Generate token → store attention K/V → reuse for next token → cache grows with context, bounded by VRAM.	vLLM PagedAttention, TensorRT-LLM, SGLang	prefill vs. decode, cache eviction, paged memory, context length, batch size limit
Flash Attention	Attention computed in fused, tiled GPU kernels → fewer HBM reads/writes → longer contexts on same hardware.	FlashAttention-2/3, xFormers, PyTorch SDPA	IO-aware, kernel fusion, tiling, memory bandwidth, long context enablement
MoE	Token → router picks top-k experts → only those FFN experts compute → sparse activation, large total capacity.	Mixtral, DeepSpeed-MoE, Megablocks, vLLM	expert routing, sparse activation, active vs. total params, load balancing loss
Distributed Training	Shard data/model across GPUs → sync gradients each step → checkpoint → scale to multi-node cluster.	DeepSpeed, FSDP, Megatron-LM, Ray Train, NCCL	data/tensor/pipeline parallelism, ZeRO, all-reduce, gradient sync, checkpointing

Sharding	Oversized model/index → partition across nodes → route request to correct shard → merge partial results.	FSDP, Milvus, Elasticsearch shards, Vitess	partition key, shard routing, rebalancing, horizontal scale, hot shard
Replication	Deploy N identical replicas behind LB → distribute traffic → survive node failure → meet availability SLA.	Kubernetes ReplicaSets, KServe, load balancers	replica count, high availability, failover, redundancy, throughput scaling
Autoscaling	Monitor queue depth/utilization → scale replicas up/down → balance GPU cost against latency SLO.	KEDA, Kubernetes HPA, Knative, cloud autoscalers	scale-to-zero, cold start, scaling policy, queue depth trigger, GPU warm pool
Vector Database	Embed chunks → build ANN index → query with filters → top-k results → ongoing reindex/backup ops.	Pinecone, Weaviate, Qdrant, Milvus, pgvector	HNSW, IVF-PQ, recall/latency trade-off, namespace, hybrid filter, reindexing
API Gateway	Client call → authenticate → rate limit → route to model/service → log usage → return response.	Kong, LiteLLM, Portkey, AWS API Gateway, Envoy	auth, quota, routing, usage metering, chargeback, central policy enforcement

Module 10 · Enterprise AI Platforms & Governance

Concept	Workflow (one line)	Tools	Techniques & Keywords
Azure AI	Provision in tenant → select model → ground on enterprise data → secure with Entra ID → deploy to apps.	Azure AI Foundry, Azure OpenAI, AI Search, Entra ID	private endpoints, Entra ID, content filters, Microsoft ecosystem integration
AWS Bedrock	Choose foundation model via one API → attach knowledge base/agents → deploy in VPC → monitor with CloudWatch.	Bedrock, Knowledge Bases, Bedrock Agents, SageMaker	multi-model API, provisioned throughput, VPC isolation, Guardrails, IAM
Vertex AI	Connect BigQuery data → train/tune or call Gemini → build RAG/agent → deploy to endpoint → monitor.	Vertex AI, Gemini, BigQuery, Agent Builder	Model Garden, grounding, AutoML, pipelines, Google Cloud data integration
LangChain	Compose loaders → splitters → retriever → prompt → LLM → parser into a chain → run and trace.	LangChain, LangSmith, community integrations	chains, LCEL, document loaders, retrievers, output parsers, abstraction cost
LangGraph	Define nodes + edges + shared state → compile graph with cycles → execute with checkpoints and interrupts.	LangGraph, LangGraph Platform, LangSmith	state graph, conditional edges, cycles, checkpointing, human interrupts
Semantic Kernel	Register typed plugins/functions → planner selects → kernel orchestrates calls → integrate into .NET app.	Semantic Kernel (C#/Python/Java), Azure AI	plugins, planners, kernel functions, typed contracts, .NET enterprise integration
CrewAI	Define agent roles + goals + tasks → assign process (sequential/hierarchical) → crew executes → combined output.	CrewAI, CrewAI Enterprise, LiteLLM	roles, goals, backstory, tasks, sequential/hierarchical process, delegation
AutoGen	Define conversable agents → agents message each other (human optional) → converge on solution → terminate.	AutoGen (AG2), Microsoft Magentic-One, Studio	conversable agents, group chat, termination condition, human proxy agent
MCP	Expose data/tool as MCP server → client discovers capabilities → model invokes over standard protocol → result returned.	MCP SDKs, Claude Desktop/Code, MCP registries	servers/clients, tools, resources, prompts, transport (stdio/HTTP), discovery
A2A	Agent publishes capability card → peer agent discovers → negotiates task → delegates → receives result.	A2A protocol (Google-initiated), agent gateways	agent card, capability discovery, task delegation, cross-vendor interoperability
Knowledge Graph	Extract entities/relations from systems → conform to ontology → load into graph → query for AI grounding.	Neo4j, TigerGraph, Amazon Neptune, RDF stores	ontology, entity resolution, Cypher/SPARQL, master data, semantic layer
Data Governance	Catalog data → classify sensitivity → assign ownership → enforce access policy → audit usage in AI systems.	Collibra, Alation, Microsoft Purview, Unity Catalog	data catalog, classification, stewardship, access policy, usage rights, quality
Enterprise Search	Connect content sources → index with permissions → semantic + keyword retrieval → grounded answer with citations.	Glean, Azure AI Search, Elastic, Amazon Q, Coveo	connectors, permission-aware indexing, federated search, citations, freshness
Data Lineage	Instrument pipelines → capture source-to-target transformations → graph the flow → trace any AI output to origin.	OpenLineage, Marquez, Purview, dbt, Unity Catalog	upstream/downstream, impact analysis, provenance, transformation tracking
Compliance	Classify use-case risk → apply required controls → document (model cards, DPIA) → evidence for regulators → re-review.	Credo AI, IBM watsonx.governance, OneTrust, ServiceNow	EU AI Act, GDPR, NIST AI RMF, ISO 42001, model cards, DPIA, risk tiers

Security	Threat-model the AI system → apply input/output and tool controls → red-team → monitor → incident response.	Llama Guard, Lakera, Robust Intelligence, Garak	prompt injection, jailbreak, data poisoning, model extraction, OWASP LLM Top 10
Identity	Authenticate user → issue scoped token → agent acts with delegated least-privilege credentials → audited per action.	Entra ID, Okta, Auth0, OAuth 2.0, SPIFFE	OAuth, on-behalf-of flow, service principal, agent identity, least privilege, RBAC
Workflow	Map existing business process → insert AI step at chosen node → keep approvals → measure cycle-time gain.	ServiceNow, Camunda, Power Automate, Temporal	BPMN, process mining, approval step, straight-through processing, SLA
Event Driven	Event published to broker → AI consumer triggered → processes asynchronously → emits result event downstream.	Kafka, EventBridge, Pub/Sub, RabbitMQ, NATS	pub/sub, event bus, async processing, backpressure, idempotent consumer, DLQ
Microservices	Split model, retrieval, and agent logic into services → independent deploy/scale → communicate over APIs.	Kubernetes, FastAPI, gRPC, Istio, API gateways	service boundaries, independent scaling, contracts, service mesh, versioning

Alphabetical Index

All 200 concepts, A–Z, with page references

A/B Testing	42	Batch Normalization	21
A2A	51	Bayes Theorem	11
Activation Function	22	BERT	26
Adam Optimizer	23	Bias	18
Adaptive RAG	33	BM25	29
Agentic RAG	33	Browser Agent	37
Anomaly Detection	7	Caching	41
API Gateway	48	Calculus	9
Artificial Intelligence	4	Canary Deployment	42
Attention	20	CatBoost	15
Audit	43	Classification	7
Autoencoder	21	Clustering	7
AutoGen	50	CNN	19
Automation	8	Code Agent	38
Autonomous Agent	37	ColBERT	30
Autoscaling	47	Combinatorics	12
AWS Bedrock	49	Compliance	52
Azure AI	49	Context Compression	31
Backpropagation	22	Convexity	11

Coordinator	35	GAN	21
Cost Optimization	41	Governance	43
CPU Inference	46	GPT	26
CRAG	32	GPU	45
CrewAI	50	Gradient Descent	22
Critic	36	Gradients	10
Cross Validation	17	Graph Theory	12
CUDA	45	GRU	20
Data Governance	51	Guardrails	40
Data Lineage	52	Hallucination Detection	40
DBSCAN	16	Hierarchical Clustering	17
Decision Support	8	Human in the Loop	37
Decision Tree	14	Hybrid Search	29
Deep Learning	4	HyDE	31
Dense Retrieval	30	Hyperparameter Tuning	17
Dependency Parsing	25	Identity	52
Differential Equations	12	Inference	6
Diffusion	22	Instruction Tuning	27
Distributed Training	47	Judge	36
Docker	44	KMeans	16
Dropout	21	KNN	15
Eigenvalues	10	Knowledge Graph	51
Enterprise Search	51	Knowledge Graph RAG	32
Entropy	11	Knowledge Representation	5
Evaluation	40	Kubernetes	44
Event Driven	53	KV Cache	46
Executor	34	LangChain	50
Expert Systems	5	LangGraph	50
FastText	25	Latency	41
Feature Engineering	17	Learning	6
Feature Selection	17	Lemmatization	24
Feedback Loop	42	LightGBM	15
Fine Tuning	28	Linear Algebra	9
Flash Attention	46	Linear Regression	16

Load Balancing	41	Positional Encoding	20
Logistic Regression	16	Prediction	6
LoRA	27	Probability	9
Loss Function	23	Prompt Engineering	27
LSTM	19	Prompt Versioning	39
Machine Learning	4	QLoRA	27
Markov Chains	11	Quantization	46
Matrices	10	Query Expansion	31
MCP	51	Random Forest	14
Memory	35	Random Variables	13
Metadata Filtering	31	Rate Limiting	42
Microservices	53	Ray	44
MoE	47	Re Ranking	32
Model Registry	39	Reasoning	5
Monitoring	41	Reasoning Agent	37
Multi Agent	35	Recommendation	7
Multi Query Retrieval	30	Recursive Retrieval	32
Multi Vector Retrieval	31	Reflection	35
Naive Bayes	15	Regression	7
NER	25	Replication	47
Neural Network	5	Research Agent	38
Numerical Methods	12	Residual Network	21
Observability	40	RLHF	27
ONNX	46	RNN	19
Optimization (Foundations)	6	RoBERTa	26
Optimization (Mathematics)	11	Rollback	42
Parent Child Retrieval	30	Router	36
PCA	16	Search Algorithms	5
PEFT	28	Security	52
PII Masking	40	Self Attention	20
Planner	34	Self RAG	32
Planning (Foundations)	6	Semantic Kernel	50
Planning Agent	37	Semantic Search	29
POS Tagging	25	Sentence Embeddings	26

Set Theory	12	Tracing	39
Sharding	47	Transformer	20
Sparse Retrieval	30	Triton	45
Statistics	10	Variance	18
Stemming	24	Vector Database	48
Supervisor	34	Vectors	9
SVM	15	Vertex AI	49
Swarm	36	Vision Transformer	22
T5	26	vLLM	45
Tensor Algebra	13	Word2Vec	25
TensorRT	45	Workflow (Agentic AI)	36
Tokenization	24	Workflow (Enterprise AI)	52
Tool Calling	35	XGBoost	14